

BÁO CÁO CHI TIẾT & MÃ NGUỒN

MODULE 2: QUẢN LÝ KHO THỰC PHẨM & AI SCANNER

Dự án: Ứng dụng Quản lý Dinh dưỡng Cá nhân với AI (Smart Nutrition)

Nền tảng: Android Native (Kotlin) + Firebase Auth & Firestore + Jetpack Compose

Người thực hiện: Thành viên 2 - Module Pantry Scanner & Inventory

Thời gian báo cáo: Tháng 7 năm 2026

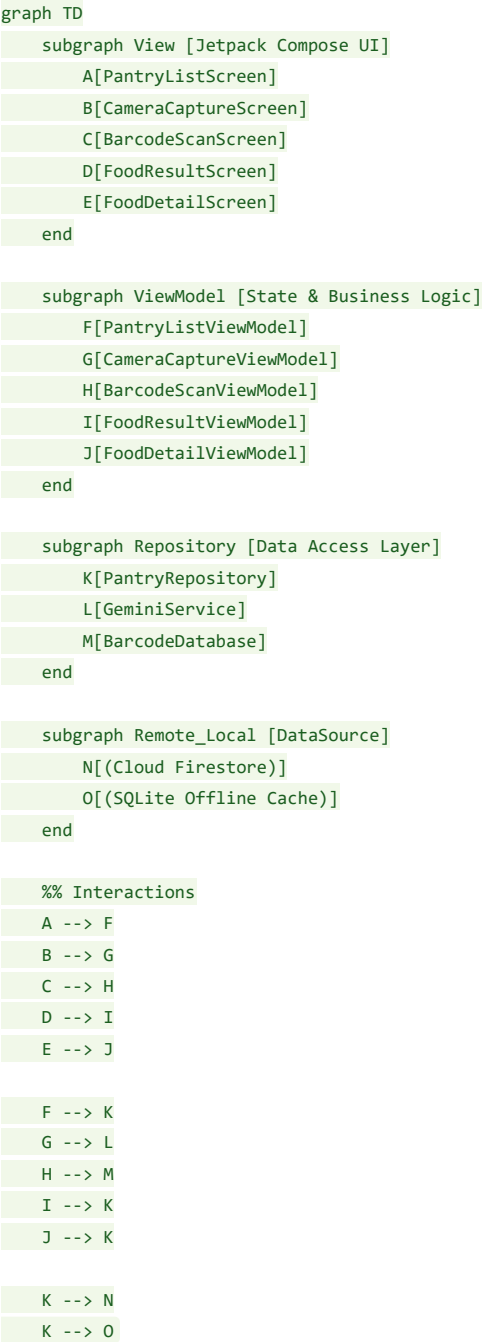
BÁO CÁO CHI TIẾT: MODULE 2 — PANTRY SCANNER & INVENTORY

1. TỔNG QUAN HỆ THỐNG & KIẾN TRÚC (MVVM)

Module 2 phụ trách **quản lý kho thực phẩm (Pantry)** của người dùng, tích hợp các công nghệ phần cứng và AI tiên tiến:

- **CameraX API:** Hỗ trợ chụp ảnh thực phẩm (Camera AI) và truyền luồng ảnh phân tích mã vạch.
- **Gemini Vision API (SDK `generativeai`):** Nhận diện hình ảnh thực phẩm tươi sống, trả về lượng Calo và Protein ước tính.
- **Google ML Kit Barcode Scanning:** Phân tích luồng hình ảnh thời gian thực để quét mã vạch EAN-13 của thực phẩm đóng hộp.
- **Firebase Firestore Offline Persistence:** Đồng bộ hóa dữ liệu kho 2 chiều thời gian thực (realtime) hỗ trợ ngoại tuyến hoàn toàn.

Kiến trúc tuân thủ nghiêm ngặt mô hình **MVVM (Model-View-ViewModel)** phối hợp với Repository Pattern:



Luồng Dữ Liệu Chi Tiết:

1. Quét mã vạch (Barcode Scan):

- BarcodeScanScreen dùng CameraX phân tích khung hình → Truyền sang ML Kit BarcodeScanner .
- Phát hiện mã vạch → BarcodeScanViewModel tra cứu trong BarcodeDatabase (Local HashMap cho Việt Nam).
- Nếu khớp: Trả về thông tin thực phẩm dưới dạng bottom sheet. Nếu không khớp: Cho phép người dùng chuyển tới FoodResultScreen để tự nhập thông tin thủ công.

2. Nhận diện hình ảnh (Camera AI):

- CameraCaptureScreen chụp ảnh bằng CameraX ImageCapture .
- CameraCaptureViewModel gửi Bitmap đã thu gọn (max 1024px) tới GeminiService .
- GeminiService gọi API gemini-2.5-flash kèm prompt chuyên biệt trả về cấu trúc JSON.
- Parse kết quả thành FoodRecognitionResult rồi truyền sang màn hình FoodResultScreen bằng savedStateHandle của Compose Navigation để người dùng kiểm tra lại và chỉnh sửa trước khi lưu.

3. Quản lý kho (Pantry List):

- `PantryListScreen` lắng nghe `Flow<List<PantryItem>>` từ `PantryRepository` thông qua Snapshot Listener của Firestore.
- Trạng thái hết hạn sử dụng (`ExpiryStatus`) được tính toán động ngay trên UI dựa vào ngày hiện tại và ngày hết hạn lưu trên document Firestore.

2. THUẬT TOÁN & LOGIC CỐT LÕI

A. Tự động Nén & Điều chỉnh Kích thước Ảnh (Image Optimization)

Để tránh lỗi vượt quá dung lượng request (thường là 4MB) của Gemini API và tiết kiệm băng thông mạng, ảnh Bitmap chụp từ camera được xử lý giảm kích thước thông qua phương thức `resizeBitmap` trong `GeminiService.kt` :

- **Nguyên lý:** Tìm hệ số tỷ lệ `ratio = minOf(1024 / width, 1024 / height)` .
- **Thực thi:** Nếu ảnh lớn hơn 1024px ở chiều rộng hoặc chiều cao, ảnh sẽ được nhân với tỉ lệ này để co nhỏ lại mà vẫn giữ nguyên tỉ lệ khung hình (Aspect Ratio), sau đó tạo một bản Bitmap mới thông qua `Bitmap.createScaledBitmap()` .

B. Kỹ thuật Kỹ sư Prompt (Prompt Engineering) với Gemini

Nhận diện hình ảnh yêu cầu dữ liệu trả về phải có cấu trúc ổn định để ứng dụng Android tự động chuyển đổi thành đối tượng Kotlin (Deserialization).

Prompt được tinh chỉnh nghiêm ngặt trong `GeminiService.kt` :

```
private const val FOOD_RECOGNITION_PROMPT = """
Bạn là chuyên gia dinh dưỡng. Nhận diện thực phẩm trong ảnh này.
Trả về ĐÚNG 1 JSON object với format sau:
{"name": "tên tiếng Việt", "calories": số kcal trên 100g, "protein": số gram protein trên 100g}
```

```
Quy tắc:
- "name": tên thực phẩm bằng tiếng Việt, viết thường, ngắn gọn
- "calories": số nguyên, ước tính kcal trên 100g
- "protein": số nguyên, ước tính gram protein trên 100g
- Chỉ trả JSON thuần, KHÔNG giải thích thêm, KHÔNG wrap trong markdown
"""
```

Xử lý Ngoại lệ (Robustness): Gemini thỉnh thoảng sẽ tự động bao bọc JSON trong khối mã Markdown (ví dụ: ````json\n...\n````). Do đó, hàm `stripMarkdownWrapper` sẽ loại bỏ các ký tự bọc này trước khi đưa chuỗi JSON vào thư viện `Gson` để parse.

C. Logic Phân loại Cảnh báo Hạn Sử Dụng (Expiry Alert Logic)

Ứng dụng thực hiện phân loại thực phẩm thành 3 mức độ cảnh báo dựa trên số ngày còn lại đến khi hết hạn sử dụng. Logic được cài đặt tại `ExpiryStatus.kt` :

- **Tính toán số ngày còn lại:**

```
$$\text{days} = \text{ChronoUnit.DAYS.between}(\text{LocalDate.now()}, \text{expiryLocalDate})$$
```

- **Quy tắc phân loại:**
-  **FRESH (Còn tươi):** $\text{days} > 3$
-  **EXPIRING (Sắp hết hạn):** $1 \leq \text{days} \leq 3$
-  **EXPIRED (Đã hết hạn):** $\text{days} \leq 0$

- **Màu sắc hiển thị:** Sử dụng mã hex màu tiêu chuẩn để đồng bộ hóa mã giao diện UI (`#4CAF50` cho xanh, `#FFC107` cho vàng, `#F44336` cho đỏ).

3. FIRESTORE DATABASE SCHEMA

Mỗi tài khoản có một kho thực phẩm riêng biệt, được tổ chức dưới dạng một **Sub-collection** nằm bên trong document của người dùng.

Path: `users/{uid}/pantry/{itemId}`

Field Name	Firestore Data Type	Description	Example / Value		
id	String (Document ID)	ID tự động sinh bởi Firestore	"zX9Y7wK2sLm8N"		
name	String	Tên thực phẩm (tiếng Việt)	"Ức gà"		
caloriesPer100g	Number (Integer)	Lượng calo trên 100g thực phẩm	165		
proteinPer100g	Number (Integer)	Lượng protein trên 100g thực phẩm	31		
quantityGrams	Number (Integer)	Khối lượng/Số lượng thực phẩm	500		
unit	String	Đơn vị đo lường	"gram" \	"piece" \	"ml"
source	String	Nguồn gốc dữ liệu nhập vào	"camera" \	"barcode" \	"manual"
imageUrl	String	Đường dẫn ảnh chụp (nếu có)	"https://firebasestorage..."		
barcode	String	Mã vạch EAN-13 nếu quét qua barcode	"8934673583220"		
expiryDate	Timestamp	Thời điểm hết hạn sử dụng	Timestamp(seconds=1782877200, nanoseconds=0)		
addedAt	Timestamp	Thời điểm đưa vào kho	Timestamp.now()		
status	String	Trạng thái đồng bộ (từ ExpiryStatus)	"fresh" \	"expiring" \	"expired"

4. CHI TIẾT CÁC LỚP TRONG MODULE 2

A. Data Layer (`com.team.smartnutrition.pantry.data`)

1. `PantryRepository.kt` :

- `addItem(uid, item)` : Thêm tài nguyên mới vào database. Ghi trực tiếp vào SQLite cache local để hỗ trợ ngoại tuyến ngay lập tức.
- `getItems(uid)` : Trả về một `Flow<List<PantryItem>>` sử dụng Snapshot Listener, sắp xếp theo thứ tự `expiryDate` tăng dần (thực phẩm sắp hết hạn đứng trước).
- `getAvailableItems(uid)` : Lấy các thực phẩm chưa bị hết hạn (cho Module 3 - AI Meal Planner lên thực đơn).
- `updateItem(...)` & `deleteItem(...)` : Chỉnh sửa thông tin/xóa thực phẩm.

2. `GeminiService.kt` :

- Gọi và quản lý kết nối tới Gemini API. Chuyển đổi tệp ảnh chụp Bitmap thành dữ liệu phân tích dạng JSON.

B. ViewModel Layer (`com.team.smartnutrition.pantry.viewmodel`)

1. `PantryListViewModel.kt` :

- Lắng nghe danh sách thực phẩm từ Repo. Thực hiện bộ lọc tìm kiếm (Search Query) và bộ lọc theo trạng thái hạn dùng (`ExpiryFilter` : ALL, EXPIRING, EXPIRED) trên bộ nhớ (in-memory) để đảm bảo tốc độ phản hồi UI cực nhanh.

2. `CameraCaptureViewModel.kt` :

- Quản lý trạng thái luồng camera (quản lý quyền camera `hasCameraPermission`) và trạng thái chờ kết quả phản hồi của AI (`isProcessing`).

3. `BarcodeScanViewModel.kt` :

- Quản lý sự kiện quét được mã vạch. Thực hiện đối chiếu mã với `BarcodeDatabase` cục bộ để tìm kiếm thông tin ngay tức thì.

4. `FoodResultViewModel.kt` :

- Lưu trữ kết quả tạm thời nhận từ camera/barcode và xử lý sự kiện bấm "Xác nhận" để lưu vào kho thực phẩm Firestore.

5. `FoodDetailViewModel.kt` :

- Quản lý thông tin chi tiết của một mặt hàng cụ thể, xử lý logic chỉnh sửa số lượng nhanh (ví dụ: tăng/giảm gam) và cập nhật ngày hết hạn mới.

C. UI Component Layer (`com.team.smartnutrition.pantry`)

1. `PantryListScreen.kt` :

- Màn hình chính. Hiển thị danh sách thực phẩm với các màu sắc trực quan (Xanh/Vàng/Đỏ). Sử dụng `SwipeToDismissBox` của Material3 để vuốt sang trái để xóa một cách trực quan.

2. `CameraCaptureScreen.kt` :

- Màn hình camera toàn cảnh bằng CameraX `PreviewView` . Nút chụp kết nối trực tiếp với SDK Gemini.

3. `BarcodeScanScreen.kt` :

- Màn hình quét mã vạch thời gian thực. Vẽ khung ngắm (Overlay Scan Window) bằng Canvas Compose và truyền luồng phân tích hình ảnh (Image Analysis) vào thư viện Google ML Kit.

4. `FoodResultScreen.kt` :

- Form điền thông tin thực phẩm (tự động điền dữ liệu trả về từ Gemini hoặc Barcode). Người dùng có thể chỉnh sửa lại tên, calo, protein, số lượng và ngày hết hạn.

5. `FoodDetailScreen.kt` :

- Màn hình chi tiết thực phẩm. Hiển thị thông số dinh dưỡng chi tiết và cho phép chỉnh sửa nhanh số lượng thực phẩm.

5. BỘ CÂU HỎI Q&A PHẢN BIỆN (DÀNH CHO BÁO CÁO THẦY CÔ)

? Câu hỏi 1: Tại sao em lại sử dụng ImageAnalysis của CameraX kết hợp ML Kit mà không chụp ảnh rồi mới quét mã vạch?

- **Trả lời:** Sử dụng `ImageAnalysis` cho phép ứng dụng phân tích trực tiếp từng khung hình trong luồng xem trước của camera (Camera Preview) với tần suất cao (real-time). Người dùng chỉ cần hướng camera vào mã vạch mà không cần bấm nút chụp, giúp cải thiện trải nghiệm người dùng tối đa. ML Kit xử lý cực nhanh và trực tiếp ngay trên thiết bị (on-device) nên không gây trễ giao diện.

? Câu hỏi 2: Gemini Vision API yêu cầu kết nối mạng, nếu app bị mất mạng đột ngột khi đang chụp ảnh thực phẩm thì xử lý thế nào?

- **Trả lời:**

1. Trong `GeminiService.kt`, em đã cài đặt khối cấu trúc `try-catch` và `withTimeoutOrNull` (30 giây) bao quanh yêu cầu gửi tới API. Nếu không có mạng hoặc quá thời gian, app sẽ ném ra lỗi `"Timeout khi gọi AI nhận diện thực phẩm"` hoặc `"Lỗi kết nối mạng"`.
2. Lỗi này được ViewModel đón nhận và lưu vào trạng thái `errorMessage` để hiển thị một thông báo Snackbar cảnh báo trực quan cho người dùng.
3. Đồng thời, ứng dụng cung cấp tùy chọn "Nhập thủ công" (Manual Entry) trên giao diện để người dùng có thể tự điền thông tin thực phẩm mà không bị gián đoạn trải nghiệm sử dụng.

? Câu hỏi 3: Nếu mã vạch quét được không có trong cơ sở dữ liệu local (BarcodeDatabase), app sẽ xử lý ra sao?

- **Trả lời:** `BarcodeDatabase` hiện tại là một HashMap cục bộ chứa các sản phẩm phổ biến ở Việt Nam. Khi quét một mã lạ, hàm `lookup` trả về `null`. Khi đó, `BarcodeScanViewModel` sẽ chuyển đổi trạng thái `showNotFoundDialog` thành `true`. Trên giao diện sẽ xuất hiện một hộp thoại thông báo sản phẩm chưa có trong hệ thống, kèm nút hướng dẫn người dùng tới màn hình nhập thông tin thủ công (với mã vạch đã được điền sẵn vào ô thông tin).

? Câu hỏi 4: Cơ chế Offline-first của kho thực phẩm hoạt động cụ thể thế nào trên Firestore?

- **Trả lời:** Khi khởi chạy ứng dụng, Firestore đã được cấu hình bật tính năng lưu trữ ngoại tuyến (`PersistenceEnabled`). Khi gọi `pantryRef(uid).document().set(data)` trong `PantryRepository`, dữ liệu sẽ được ghi ngay lập tức vào cơ sở dữ liệu SQLite cục bộ trên điện thoại và báo thành công cho UI cập nhật danh sách lập tức. Khi thiết bị có mạng trở lại, Firebase SDK sẽ tự động chạy tiến trình ngầm đồng bộ hóa dữ liệu này lên Cloud Firestore ở trên đám mây.

? Câu hỏi 5: Tại sao em lại giới hạn kích thước ảnh gửi lên Gemini là 1024px? Tại sao không dùng ảnh gốc chất lượng cao cho chính xác?

- **Trả lời:** Camera điện thoại hiện nay chụp ảnh có độ phân giải rất lớn (thường > 12 Megapixels, kích thước file > 5MB). Nếu gửi trực tiếp ảnh gốc này:

1. Dễ gặp lỗi tràn bộ nhớ (OutOfMemoryError) trên các dòng máy Android có dung lượng RAM hạn chế.
2. Vượt quá giới hạn gói tin tải lên của Gemini Vision API.
3. Tốc độ tải lên mạng sẽ rất chậm, gây khó chịu cho người dùng.

Qua thử nghiệm thực tế, độ phân giải 1024px là hoàn toàn đủ chi tiết để các mô hình thị giác máy tính như `gemini-2.5-flash` nhận diện chính xác các loại thực phẩm thông thường, trong khi dung lượng tệp tin giảm đi hơn 90% (chỉ còn khoảng vài trăm KB).

MÃ NGUỒN CHI TIẾT MODULE 2 (SOURCE CODE)

Dưới đây là nội dung toàn bộ mã nguồn của các lớp thuộc Module 2, sắp xếp theo cấu trúc thư mục dự án.

File: pantry/BarcodeScanScreen.kt


```

package com.team.smartnutrition.pantry

import android.Manifest
import android.content.pm.PackageManager
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.camera.core.CameraSelector
import androidx.camera.core.ExperimentalGetImage
import androidx.camera.core.ImageAnalysis
import androidx.camera.core.ImageProxy
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView
import androidx.concurrent.futures.await
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.BlendMode
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalLifecycleOwner
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.core.content.ContextCompat
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.google.mlkit.vision.barcode.BarcodeScannerOptions
import com.google.mlkit.vision.barcode.BarcodeScanning
import com.google.mlkit.vision.barcode.common.Barcode
import com.google.mlkit.vision.common.InputImage
import com.team.smartnutrition.common.components.GradientButton
import com.team.smartnutrition.navigation.Screen
import com.team.smartnutrition.pantry.viewmodel.BarcodeScanViewModel

/**
 * =====
 * MODULE 2 - TV2: QUÉT MÃ VẠCH
 * =====
 *
 * Camera quét barcode real-time bằng ML Kit.
 * Phát hiện barcode → tra cứu BarcodeDatabase → navigate FoodResult.
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun BarcodeScanScreen(
    navController: NavController,
    viewModel: BarcodeScanViewModel = viewModel()
) {
    val context = LocalContext.current

```

```

val lifecycleOwner = LocalLifecycleOwner.current
val uiState by viewModel.uiState.collectAsState()

// == PERMISSION ==
val permissionLauncher = rememberLauncherForActivityResult(
    ActivityResultContracts.RequestPermission()
) { granted ->
    viewModel.updatePermission(granted)
}

LaunchedEffect(Unit) {
    val hasPermission = ContextCompat.checkSelfPermission(
        context, Manifest.permission.CAMERA
    ) == PackageManager.PERMISSION_GRANTED

    if (hasPermission) {
        viewModel.updatePermission(true)
    } else {
        permissionLauncher.launch(Manifest.permission.CAMERA)
    }
}

// == NAVIGATION EFFECT ==
LaunchedEffect(uiState.navigateForResult) {
    if (uiState.navigateForResult) {
        navController.currentBackStackEntry?.savedStateHandle?.apply {
            if (uiState.resultJson != null) {
                set("food_result_json", uiState.resultJson)
            }
            set("food_source", "barcode")
            set("food_barcode", uiState.detectedBarcode ?: "")
        }
        navController.navigate(Screen.FoodResult.route)
        viewModel.onNavigated()
    }
}

// == UI ==
if (!uiState.hasCameraPermission) {
    // Reuse permission denied pattern
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Column(
            modifier = Modifier.padding(32.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Icon(
                Icons.Filled.QrCodeScanner,
                contentDescription = null,
                modifier = Modifier.size(80.dp),
                tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.5f)
            )
            Spacer(modifier = Modifier.height(24.dp))
            Text(
                text = "Cần quyền truy cập Camera",
                style = MaterialTheme.typography.titleLarge,
                textAlign = TextAlign.Center
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text(

```

```

        text = "Ứng dụng cần quyền camera để quét mã vạch sản phẩm.",
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        textAlign = TextAlign.Center
    )
    Spacer(modifier = Modifier.height(24.dp))
    Button(onClick = { permissionLauncher.launch(Manifest.permission.CAMERA) }) {
        Text("Cấp quyền Camera")
    }
    Spacer(modifier = Modifier.height(12.dp))
    TextButton(onClick = { navController.popBackStack() }) {
        Text("Quay lại")
    }
}
}
} else {
    Box(modifier = Modifier.fillMaxSize()) {
        // Camera Preview with Barcode Analyzer
        val previewView = remember { PreviewView(context) }

        // Setup camera with barcode analyzer
        LaunchedEffect(uiState.hasCameraPermission) {
            if (!uiState.hasCameraPermission) return@LaunchedEffect
            try {
                val cameraProvider = ProcessCameraProvider.getInstance(context).await()
                val preview = Preview.Builder().build().also {
                    it.setSurfaceProvider(previewView.surfaceProvider)
                }
                val imageAnalysis = ImageAnalysis.Builder()
                    .setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)
                    .build()
                    .also {
                        it.setAnalyzer(
                            ContextCompat.getMainExecutor(context),
                            BarcodeAnalyzer { rawValue ->
                                viewModel.onBarcodeDetected(rawValue)
                            }
                        )
                    }
                cameraProvider.unbindAll()
                cameraProvider.bindToLifecycle(
                    lifecycleOwner,
                    CameraSelector.DEFAULT_BACK_CAMERA,
                    preview,
                    imageAnalysis
                )
            } catch (_: Exception) {}
        }
    }

    AndroidView(
        factory = { previewView },
        modifier = Modifier.fillMaxSize()
    )

    // Scan frame overlay
    ScanFrameOverlay()

    // Top bar
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .statusBarsPadding()

```

```

        .padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        IconButton(
            onClick = { navController.popBackStack() },
            colors = IconButtonDefaults.iconButtonColors(
                containerColor = Color.Black.copy(alpha = 0.5f),
                contentColor = Color.White
            )
        ) {
            Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Quay lại")
        }
        Spacer(modifier = Modifier.weight(1f))
        Text(
            text = "Quét mã vạch",
            style = MaterialTheme.typography.titleMedium,
            color = Color.White,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.weight(1f))
        Spacer(modifier = Modifier.size(48.dp))
    }
}

```

```

// Instruction text
Text(
    text = "Hướng camera vào mã vạch sản phẩm",
    style = MaterialTheme.typography.bodyLarge,
    color = Color.White,
    fontWeight = FontWeight.Medium,
    modifier = Modifier
        .align(Alignment.BottomCenter)
        .padding(bottom = 80.dp)
        .background(
            Color.Black.copy(alpha = 0.5f),
            RoundedCornerShape(20.dp)
        )
        .padding(horizontal = 24.dp, vertical = 12.dp)
)

```

```

// Loading overlay
if (uiState.isLookingUp) {
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.Black.copy(alpha = 0.7f)),
        contentAlignment = Alignment.Center
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            CircularProgressIndicator(color = Color.White)
            Spacer(modifier = Modifier.height(16.dp))
            Text("Đang tra cứu sản phẩm...", color = Color.White)
        }
    }
}
}
}
}

```

```

// == RESULT BOTTOM SHEET ==
if (uiState.showResultSheet && uiState.lookupResult != null) {
    ModalBottomSheet(
        onDismissRequest = { viewModel.dismissResultSheet() }
    ) {

```

```

        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(24.dp)
        ) {
            Text(
                text = "👍 Sản phẩm tìm thấy",
                style = MaterialTheme.typography.titleLarge,
                fontWeight = FontWeight.Bold,
                color = MaterialTheme.colorScheme.primary
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = "Mã vạch: ${uiState.detectedBarcode}",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
            Spacer(modifier = Modifier.height(20.dp))

            Card(
                modifier = Modifier.fillMaxWidth(),
                shape = RoundedCornerShape(16.dp),
                colors = CardDefaults.cardColors(
                    containerColor = MaterialTheme.colorScheme.surfaceVariant
                )
            ) {
                Column(modifier = Modifier.padding(20.dp)) {
                    Text(
                        text = uiState.lookupResult!!.name,
                        style = MaterialTheme.typography.headlineSmall,
                        fontWeight = FontWeight.Bold
                    )
                    Spacer(modifier = Modifier.height(12.dp))
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.SpaceEvenly
                    ) {
                        Column(horizontalAlignment = Alignment.CenterHorizontally) {
                            Text(
                                text = "${uiState.lookupResult!!.calories}",
                                style = MaterialTheme.typography.headlineMedium,
                                fontWeight = FontWeight.Bold,
                                color = MaterialTheme.colorScheme.primary
                            )
                            Text("kcal/100g", style = MaterialTheme.typography.bodySmall)
                        }
                        Column(horizontalAlignment = Alignment.CenterHorizontally) {
                            Text(
                                text = "${uiState.lookupResult!!.protein}g",
                                style = MaterialTheme.typography.headlineMedium,
                                fontWeight = FontWeight.Bold,
                                color = MaterialTheme.colorScheme.tertiary
                            )
                            Text("protein/100g", style = MaterialTheme.typography.bodySmall)
                        }
                    }
                }
            }

            Spacer(modifier = Modifier.height(20.dp))

            GradientButton(

```

```

        text = "Thêm vào kho",
        onClick = { viewModel.confirmResult() },
        icon = Icons.Filled.Add
    )

    Spacer(modifier = Modifier.height(12.dp))

    TextButton(
        onClick = { viewModel.resetScan() },
        modifier = Modifier.fillMaxWidth()
    ) { Text("Quét mã khác") }

    Spacer(modifier = Modifier.height(16.dp))
}

// == NOT FOUND DIALOG ==
if (uiState.showNotFoundDialog) {
    AlertDialog(
        onDismissRequest = { viewModel.dismissNotFoundDialog() },
        icon = { Icon(Icons.Filled.SearchOff, contentDescription = null) },
        title = { Text("Không tìm thấy sản phẩm") },
        text = {
            Text("Mã vạch: ${uiState.detectedBarcode}\n\nSản phẩm này chưa có trong cơ sở dữ liệu. Bạn có muốn nhập thông tin thủ công?")
        },
        confirmButton = {
            TextButton(onClick = { viewModel.navigateManualEntry() }) {
                Text("Nhập thủ công")
            }
        },
        dismissButton = {
            TextButton(onClick = { viewModel.dismissNotFoundDialog() }) {
                Text("Quét lại")
            }
        }
    )
}

/**
 * Scan frame overlay: semi-transparent background + scan window ở giữa.
 */
@Composable
private fun ScanFrameOverlay() {
    Canvas(modifier = Modifier.fillMaxSize()) {
        val scanWidth = size.width * 0.75f
        val scanHeight = 200f
        val left = (size.width - scanWidth) / 2
        val top = (size.height - scanHeight) / 2

        // Semi-transparent overlay
        // Top
        drawRect(
            color = Color.Black.copy(alpha = 0.5f),
            topLeft = Offset.Zero,
            size = Size(size.width, top)
        )
        // Bottom
        drawRect(
            color = Color.Black.copy(alpha = 0.5f),
            topLeft = Offset(0f, top + scanHeight),

```

```

        size = Size(size.width, size.height - top - scanHeight)
    )
    // Left
    drawRect(
        color = Color.Black.copy(alpha = 0.5f),
        topLeft = Offset(0f, top),
        size = Size(left, scanHeight)
    )
    // Right
    drawRect(
        color = Color.Black.copy(alpha = 0.5f),
        topLeft = Offset(left + scanWidth, top),
        size = Size(size.width - left - scanWidth, scanHeight)
    )

    // Corner lines
    val cornerLength = 30f
    val cornerColor = Color(0xFF4CAF50)
    val strokeWidth = 4f

    // Top-left corner
    drawLine(cornerColor, Offset(left, top), Offset(left + cornerLength, top), strokeWidth)
    drawLine(cornerColor, Offset(left, top), Offset(left, top + cornerLength), strokeWidth)
    // Top-right corner
    drawLine(cornerColor, Offset(left + scanWidth - cornerLength, top), Offset(left + scanWidth, top), strokeWidth)
    drawLine(cornerColor, Offset(left + scanWidth, top), Offset(left + scanWidth, top + cornerLength), strokeWidth)
    // Bottom-left corner
    drawLine(cornerColor, Offset(left, top + scanHeight - cornerLength), Offset(left, top + scanHeight), strokeWidth)
    drawLine(cornerColor, Offset(left, top + scanHeight), Offset(left + cornerLength, top + scanHeight), strokeWidth)
    // Bottom-right corner
    drawLine(cornerColor, Offset(left + scanWidth, top + scanHeight - cornerLength), Offset(left + scanWidth, top + scanHeight), strokeWidth)
    drawLine(cornerColor, Offset(left + scanWidth - cornerLength, top + scanHeight), Offset(left + scanWidth, top + scanHeight), strokeWidth)
    }
}

/**
 * ML Kit Barcode Analyzer cho CameraX ImageAnalysis.
 */
private class BarcodeAnalyzer(
    private val onBarcodeDetected: (String) -> Unit
) : ImageAnalysis.Analyzer {

    private val scanner = BarcodeScanning.getClient(
        BarcodeScannerOptions.Builder()
            .setBarcodeFormats(
                Barcode.FORMAT_EAN_13,
                Barcode.FORMAT_EAN_8,
                Barcode.FORMAT_UPC_A
            )
            .build()
    )

    @androidx.annotation.OptIn(ExperimentalGetImage::class)
    override fun analyze(imageProxy: ImageProxy) {
        val mediaImage = imageProxy.image
        if (mediaImage == null) {
            imageProxy.close()
            return
        }

        val inputImage = InputImage.fromMediaImage(
            mediaImage, imageProxy.imageInfo.rotationDegrees

```

```
    )  
  
    scanner.process(inputImage)  
        .addOnSuccessListener { barcodes ->  
            barcodes.firstOrNull()?.rawValue?.let { value ->  
                onBarcodeDetected(value)  
            }  
        }  
        .addOnCompleteListener {  
            imageProxy.close()  
        }  
    }  
}
```

File: pantry/CameraCaptureScreen.kt


```

package com.team.smartnutrition.pantry

import android.Manifest
import android.content.pm.PackageManager
import android.graphics.Bitmap
import android.graphics.Matrix
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.camera.core.CameraSelector
import androidx.camera.core.ImageCapture
import androidx.camera.core.ImageCaptureException
import androidx.camera.core.ImageProxy
import androidx.camera.core.Preview
import androidx.camera.lifecycle.ProcessCameraProvider
import androidx.camera.view.PreviewView
import androidx.concurrent.futures.await
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalLifecycleOwner
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.core.content.ContextCompat
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.team.smartnutrition.navigation.Screen
import com.team.smartnutrition.pantry.viewmodel.CameraCaptureViewModel

/**
 * =====
 * MODULE 2 - TV2: CHỤP ẢNH THỰC PHẨM
 * =====
 *
 * CameraX Preview toàn màn hình + nút chụp.
 * Sau khi chụp → gửi Gemini Vision API → navigate FoodResult.
 */
@Composable
fun CameraCaptureScreen(
    navController: NavController,
    viewModel: CameraCaptureViewModel = viewModel()
) {
    val context = LocalContext.current
    val lifecycleOwner = LocalLifecycleOwner.current
    val uiState by viewModel.uiState.collectAsState()

    // ImageCapture use case
    val imageCapture = remember {
        ImageCapture.Builder()
    }

```

```

        .setCaptureMode(ImageCapture.CAPTURE_MODE_MINIMIZE_LATENCY)
        .build()
    }

    // == PERMISSION HANDLING ==
    val permissionLauncher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestPermission()
    ) { granted ->
        viewModel.updatePermission(granted)
    }

    LaunchedEffect(Unit) {
        val hasPermission = ContextCompat.checkSelfPermission(
            context, Manifest.permission.CAMERA
        ) == PackageManager.PERMISSION_GRANTED

        if (hasPermission) {
            viewModel.updatePermission(true)
        } else {
            permissionLauncher.launch(Manifest.permission.CAMERA)
        }
    }

    // == NAVIGATION EFFECT ==
    LaunchedEffect(uiState.navigateToResult) {
        if (uiState.navigateToResult && uiState.resultJson != null) {
            navController.currentBackStackEntry?.savedStateHandle?.apply {
                set("food_result_json", uiState.resultJson)
                set("food_source", "camera")
            }
            navController.navigate(Screen.FoodResult.route)
            viewModel.onNavigated()
        }
    }

    // == UI ==
    if (!uiState.hasCameraPermission) {
        // Permission denied state
        PermissionDeniedScreen(
            onBack = { navController.popBackStack() },
            onRequestPermission = {
                permissionLauncher.launch(Manifest.permission.CAMERA)
            }
        )
    } else {
        Box(modifier = Modifier.fillMaxSize()) {
            // Camera Preview
            val previewView = remember { PreviewView(context) }

            // Setup camera in LaunchedEffect to use await()
            LaunchedEffect(uiState.hasCameraPermission) {
                if (!uiState.hasCameraPermission) return@LaunchedEffect
                try {
                    val cameraProvider = ProcessCameraProvider.getInstance(context).await()
                    val preview = Preview.Builder().build().also {
                        it.setSurfaceProvider(previewView.surfaceProvider)
                    }
                    cameraProvider.unbindAll()
                    cameraProvider.bindToLifecycle(
                        lifecycleOwner,
                        CameraSelector.DEFAULT_BACK_CAMERA,
                        preview,

```

```

        imageCapture
    )
} catch (_: Exception) {
    // Camera binding failed
}
}

AndroidView(
    factory = { previewView },
    modifier = Modifier.fillMaxSize()
)

// Top bar overlay
Row(
    modifier = Modifier
        .fillMaxWidth()
        .statusBarsPadding()
        .padding(16.dp),
    verticalAlignment = Alignment.CenterVertically
) {
    IconButton(
        onClick = { navController.popBackStack() },
        colors = IconButtonDefaults.iconButtonColors(
            containerColor = Color.Black.copy(alpha = 0.5f),
            contentColor = Color.White
        )
    )
} {
    Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Quay lại")
}
Spacer(modifier = Modifier.weight(1f))
Text(
    text = "Chụp ảnh thực phẩm",
    style = MaterialTheme.typography.titleMedium,
    color = Color.White,
    fontWeight = FontWeight.Bold
)
Spacer(modifier = Modifier.weight(1f))
// Placeholder to balance layout
Spacer(modifier = Modifier.size(48.dp))
}

// Bottom controls
Row(
    modifier = Modifier
        .fillMaxWidth()
        .align(Alignment.BottomCenter)
        .padding(bottom = 48.dp),
    horizontalArrangement = Arrangement.SpaceEvenly,
    verticalAlignment = Alignment.CenterVertically
) {
    // Spacer left
    Spacer(modifier = Modifier.size(56.dp))

    // Capture button
    IconButton(
        onClick = {
            if (!uiState.isProcessing) {
                capturePhoto(imageCapture, context) { bitmap ->
                    viewModel.onPhotoCaptured(bitmap)
                }
            }
        }
    ),

```

```

        modifier = Modifier
            .size(72.dp)
            .border(4.dp, Color.White, CircleShape),
        colors = IconButtonDefaults.iconButtonColors(
            containerColor = Color.White.copy(alpha = 0.3f)
        )
    ) {
        Surface(
            modifier = Modifier.size(56.dp),
            shape = CircleShape,
            color = Color.White
        ) {}
    }
}

// Barcode button
IconButton(
    onClick = { navController.navigate(Screen.BarcodeScan.route) },
    modifier = Modifier.size(56.dp),
    colors = IconButtonDefaults.iconButtonColors(
        containerColor = Color.Black.copy(alpha = 0.5f),
        contentColor = Color.White
    )
) {
    Icon(
        Icons.Filled.QrCodeScanner,
        contentDescription = "Quét mã vạch",
        modifier = Modifier.size(28.dp)
    )
}
}

// Processing overlay
if (uiState.isProcessing) {
    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.Black.copy(alpha = 0.7f)),
        contentAlignment = Alignment.Center
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            CircularProgressIndicator(
                color = Color.White,
                modifier = Modifier.size(48.dp)
            )
            Spacer(modifier = Modifier.height(16.dp))
            Text(
                text = "🤖 AI đang nhận diện...",
                color = Color.White,
                style = MaterialTheme.typography.titleMedium
            )
        }
    }
}

// Error snackbar
if (uiState.errorMessage != null) {
    Snackbar(
        modifier = Modifier
            .align(Alignment.BottomCenter)
            .padding(16.dp),
        action = {
            TextButton(onClick = { viewModel.clearError() }) {

```

```

        Text("Đóng", color = Color.White)
    }
},
    containerColor = MaterialTheme.colorScheme.errorContainer,
    contentColor = MaterialTheme.colorScheme.onErrorContainer
) {
    Text(uiState.errorMessage!!)
}
}
}
}

/**
 * Chụp ảnh từ CameraX ImageCapture.
 */
private fun capturePhoto(
    imageCapture: ImageCapture,
    context: android.content.Context,
    onCaptured: (Bitmap) -> Unit
) {
    imageCapture.takePicture(
        ContextCompat.getMainExecutor(context),
        object : ImageCapture.OnImageCapturedCallback() {
            override fun onCaptureSuccess(imageProxy: ImageProxy) {
                val bitmap = imageProxy.toBitmap()
                // Xoay ảnh theo rotation nếu cần
                val rotatedBitmap = rotateBitmap(bitmap, imageProxy.imageInfo.rotationDegrees)
                imageProxy.close()
                onCaptured(rotatedBitmap)
            }
        }
    )

    override fun onError(exception: ImageCaptureException) {
        // Lỗi sẽ được hiện qua error state
    }
}

/**
 * Xoay bitmap theo rotationDegrees từ ImageProxy.
 */
private fun rotateBitmap(bitmap: Bitmap, rotationDegrees: Int): Bitmap {
    if (rotationDegrees == 0) return bitmap

    val matrix = Matrix().apply {
        postRotate(rotationDegrees.toFloat())
    }
    return Bitmap.createBitmap(bitmap, 0, 0, bitmap.width, bitmap.height, matrix, true)
}

/**
 * Màn hình khi quyền camera bị từ chối.
 */
@Composable
private fun PermissionDeniedScreen(
    onBack: () -> Unit,
    onRequestPermission: () -> Unit
) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    )
}

```

```
    ) {  
        Column(  
            modifier = Modifier.padding(32.dp),  
            horizontalAlignment = Alignment.CenterHorizontally  
        ) {  
            Icon(  
                Icons.Filled.CameraAlt,  
                contentDescription = null,  
                modifier = Modifier.size(80.dp),  
                tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.5f)  
            )  
            Spacer(modifier = Modifier.height(24.dp))  
            Text(  
                text = "Cần quyền truy cập Camera",  
                style = MaterialTheme.typography.titleLarge,  
                textAlign = TextAlign.Center  
            )  
            Spacer(modifier = Modifier.height(8.dp))  
            Text(  
                text = "Ứng dụng cần quyền camera để chụp ảnh thực phẩm và nhận diện bằng AI.",  
                style = MaterialTheme.typography.bodyMedium,  
                color = MaterialTheme.colorScheme.onSurfaceVariant,  
                textAlign = TextAlign.Center  
            )  
            Spacer(modifier = Modifier.height(24.dp))  
            Button(onClick = onRequestPermission) {  
                Text("Cấp quyền Camera")  
            }  
            Spacer(modifier = Modifier.height(12.dp))  
            TextButton(onClick = onBackPressed) {  
                Text("Quay lại")  
            }  
        }  
    }  
}
```

File: pantry/FoodDetailScreen.kt

```
package com.team.smartnutrition.pantry
```

```
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.team.smartnutrition.common.components.ErrorCard
import com.team.smartnutrition.common.components.LoadingScreen
import com.team.smartnutrition.common.components.SmartTopBar
import com.team.smartnutrition.pantry.util.ExpiryStatus
import com.team.smartnutrition.pantry.util.calculateExpiryStatus
import com.team.smartnutrition.pantry.util.daysUntilExpiry
import com.team.smartnutrition.pantry.util.expiryDisplayText
import com.team.smartnutrition.pantry.viewmodel.FoodDetailViewModel
import java.text.SimpleDateFormat
import java.util.*
```

```
/**
 * =====
 * MODULE 2 - TV2: CHI TIẾT THỰC PHẨM
 * =====
 *
 *
 * Hiển thị chi tiết 1 item trong kho:
 * - Thông tin dinh dưỡng (tên, calo, protein)
 * - Thông tin kho (số lượng, đơn vị, nguồn, mã vạch, ngày thêm, hạn SD)
 * - Badge trạng thái hạn sử dụng
 * - Edit mode cho số lượng
 * - Delete với confirm dialog
 */
```

```
@Composable
```

```
fun FoodDetailScreen(
    navController: NavController,
    itemId: String,
    viewModel: FoodDetailViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
```

```
// Load item khi mở screen
```

```
LaunchedEffect(itemId) {
    viewModel.loadItem(itemId)
}
```

```
// Navigate back sau khi xóa
```

```
LaunchedEffect(uiState.deleteSuccess) {
    if (uiState.deleteSuccess) {
        navController.popBackStack()
    }
}
```

```

Scaffold(
  topBar = {
    SmartTopBar(
      title = "Chi tiết thực phẩm",
      onBackPressed = { navController.popBackStack() },
      actions = {
        if (uiState.item != null && !uiState.isEditing) {
          // Edit button
          IconButton(onClick = { viewModel.startEditing() }) {
            Icon(Icons.Filled.Edit, contentDescription = "Sửa")
          }
          // Delete button
          IconButton(onClick = { viewModel.showDeleteConfirm() }) {
            Icon(
              Icons.Filled.Delete,
              contentDescription = "Xóa",
              tint = MaterialTheme.colorScheme.error
            )
          }
        }
        if (uiState.isEditing) {
          // Save button
          IconButton(onClick = { viewModel.saveChanges() }) {
            Icon(
              Icons.Filled.Check,
              contentDescription = "Lưu",
              tint = MaterialTheme.colorScheme.primary
            )
          }
          // Cancel button
          IconButton(onClick = { viewModel.cancelEditing() }) {
            Icon(Icons.Filled.Close, contentDescription = "Hủy")
          }
        }
      }
    )
  }
) { paddingValues ->
  when {
    uiState.isLoading -> {
      LoadingScreen(
        message = "Đang tải...",
        modifier = Modifier.padding(paddingValues)
      )
    }
    uiState.item == null -> {
      ErrorCard(
        message = uiState.errorMessage ?: "Không tìm thấy thực phẩm",
        onRetry = { viewModel.loadItem(itemId) },
        modifier = Modifier.padding(paddingValues)
      )
    }
    else -> {
      val item = uiState.item!!
      val expiryStatus = calculateExpiryStatus(item.expiryDate)
      val days = daysUntilExpiry(item.expiryDate)
      val dateFormat = SimpleDateFormat("dd/MM/yyyy", Locale.getDefault())

      Column(
        modifier = Modifier
          .fillMaxSize()
          .padding(paddingValues)

```



```

        .verticalScroll(rememberScrollState())
        .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // == STATUS BADGE (full width) ==
        Surface(
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(16.dp),
            color = Color(android.graphics.Color.parseColor(expiryStatus.colorHex))
        ) {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(16.dp),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.Center
            ) {
                Icon(
                    imageVector = when (expiryStatus) {
                        ExpiryStatus.FRESH -> Icons.Filled.CheckCircle
                        ExpiryStatus.EXPIRING -> Icons.Filled.Warning
                        ExpiryStatus.EXPIRED -> Icons.Filled.Error
                    },
                    contentDescription = null,
                    tint = Color.White
                )
                Spacer(modifier = Modifier.width(8.dp))
                Text(
                    text = expiryDisplayText(item.expiryDate),
                    style = MaterialTheme.typography.titleMedium,
                    color = Color.White,
                    fontWeight = FontWeight.Bold
                )
            }
        }
    }
}

```

```

        // == CARD 1: Thông tin dinh dưỡng ==
        Card(
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(16.dp)
        ) {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(20.dp)
            ) {
                Text(
                    text = item.name,
                    style = MaterialTheme.typography.headlineSmall,
                    fontWeight = FontWeight.Bold
                )
                Spacer(modifier = Modifier.height(16.dp))
                HorizontalDivider()
                Spacer(modifier = Modifier.height(16.dp))

                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceEvenly
                ) {
                    NutrientInfo(
                        label = "Calo",
                        value = "${item.caloriesPer100g}",

```

```

        unit = "kcal/100g",
        color = MaterialTheme.colorScheme.primary
    )
    NutrientInfo(
        label = "Protein",
        value = "${item.proteinPer100g}",
        unit = "g/100g",
        color = MaterialTheme.colorScheme.tertiary
    )
    }
}

}

// == CARD 2: Thông tin kho ==
Card(
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(16.dp)
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(20.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Icon(
                Icons.Filled.Inventory2,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.primary
            )
            Spacer(modifier = Modifier.width(8.dp))
            Text(
                text = "Thông tin kho",
                style = MaterialTheme.typography.titleMedium,
                fontWeight = FontWeight.Bold
            )
        }
    }

    HorizontalDivider()

    // Số lượng (editable khi isEditing)
    if (uiState.isEditing) {
        OutlinedTextField(
            value = uiState.editQuantity,
            onChange = { viewModel.updateEditQuantity(it) },
            label = { Text("Số lượng") },
            suffix = { Text(item.unit) },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            singleLine = true,
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(12.dp)
        )
    } else {
        DetailRow(
            icon = Icons.Filled.Scale,
            label = "Số lượng",
            value = "${item.quantityGrams} ${item.unit}"
        )
    }

    DetailRow(
        icon = when (item.source) {

```

```

        "camera" -> Icons.Filled.CameraAlt
        "barcode" -> Icons.Filled.QrCodeScanner
        else -> Icons.Filled.Edit
    },
    label = "Nguồn",
    value = when (item.source) {
        "camera" -> "📷 Chụp ảnh AI"
        "barcode" -> "📄 Quét mã vạch"
        else -> "✍️ Nhập thủ công"
    }
)

if (item.barcode.isNotBlank()) {
    DetailRow(
        icon = Icons.Filled.QrCode,
        label = "Mã vạch",
        value = item.barcode
    )
}

DetailRow(
    icon = Icons.Filled.CalendarToday,
    label = "Ngày thêm",
    value = item.addedAt?.toDate()?.let { dateFormat.format(it) } ?: "-"
)

DetailRow(
    icon = Icons.Filled.EventBusy,
    label = "Hạn sử dụng",
    value = item.expiryDate?.toDate()?.let { dateFormat.format(it) } ?: "-"
)
}

// === ERROR MESSAGE ===
if (uiState.errorMessage != null) {
    Card(
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.errorContainer
        ),
        shape = RoundedCornerShape(12.dp)
    ) {
        Text(
            text = "⚠️ ${uiState.errorMessage}",
            modifier = Modifier.padding(16.dp),
            color = MaterialTheme.colorScheme.onErrorContainer
        )
    }
}

Spacer(modifier = Modifier.height(16.dp))
}
}
}

// === DELETE CONFIRM DIALOG ===
if (uiState.showDeleteDialog) {
    AlertDialog(
        onDismissRequest = { viewModel.dismissDeleteConfirm() },
        icon = { Icon(Icons.Filled.Delete, contentDescription = null, tint = MaterialTheme.colorScheme.error) },
        title = { Text("Xóa thực phẩm") },

```

```

        text = {
            Text("Bạn có chắc muốn xóa \("${uiState.item?.name}\\" khỏi kho?")
        },
        confirmButton = {
            TextButton(
                onClick = { viewModel.deleteItem() },
                colors = ButtonDefaults.textButtonColors(
                    contentColor = MaterialTheme.colorScheme.error
                )
            ) { Text("Xóa") }
        },
        dismissButton = {
            TextButton(onClick = { viewModel.dismissDeleteConfirm() }) { Text("Hủy") }
        }
    )
}

```

```

// =====
// SUB-COMPOSABLES
// =====

```

```

@Composable
private fun NutrientInfo(
    label: String,
    value: String,
    unit: String,
    color: Color
) {
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(
            text = value,
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            color = color
        )
        Text(
            text = label,
            style = MaterialTheme.typography.bodyMedium,
            fontWeight = FontWeight.Medium
        )
        Text(
            text = unit,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}

```

```

@Composable
private fun DetailRow(
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    label: String,
    value: String
) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null,

```

```
        modifier = Modifier.size(20.dp),
        tint = MaterialTheme.colorScheme.onSurfaceVariant
    )
    Spacer(modifier = Modifier.width(12.dp))
    Text(
        text = label,
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        modifier = Modifier.width(100.dp)
    )
    Text(
        text = value,
        style = MaterialTheme.typography.bodyMedium,
        fontWeight = FontWeight.Medium
    )
}
}
```

File: pantry/FoodResultScreen.kt

```

package com.team.smartnutrition.pantry

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.team.smartnutrition.common.components.GradientButton
import com.team.smartnutrition.common.components.SmartTopBar
import com.team.smartnutrition.navigation.Screen
import com.team.smartnutrition.pantry.viewmodel.FoodResultViewModel
import java.time.Instant
import java.time.ZoneOffset
import java.time.format.DateTimeFormatter

/**
 * =====
 * MODULE 2 - TV2: KẾT QUẢ NHẬN DIỆN AI
 * =====
 *
 *
 * Hiển thị kết quả từ Gemini / barcode / empty (manual):
 * - Pre-fill tên, calo, protein
 * - Cho user chỉnh sửa
 * - Chọn số lượng, đơn vị, ngày hết hạn
 * - Nút "Lưu vào kho" → Firestore
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun FoodResultScreen(
    navController: NavController,
    viewModel: FoodResultViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    var showDatePicker by remember { mutableStateOf(false) }
    var showUnitDropdown by remember { mutableStateOf(false) }

    // === INIT FROM PREVIOUS SCREEN ===
    val foodResultJson = navController.previousBackStackEntry
        ?.savedStateHandle?.get<String>("food_result_json")
    val foodSource = navController.previousBackStackEntry
        ?.savedStateHandle?.get<String>("food_source")
    val foodBarcode = navController.previousBackStackEntry
        ?.savedStateHandle?.get<String>("food_barcode")

    LaunchedEffect(Unit) {
        viewModel.initFromResult(foodResultJson, foodSource, foodBarcode)
    }

    // === SAVE SUCCESS → NAVIGATE BACK ===
    LaunchedEffect(uiState.saveSuccess) {
        if (uiState.saveSuccess) {

```

```

        navController.popBackStack(Screen.Pantry.route, inclusive = false)
    }
}

Scaffold(
    topBar = {
        SmartTopBar(
            title = when (uiState.source) {
                "camera" -> "📷 Kết quả AI"
                "barcode" -> "📦 Sản phẩm barcode"
                else -> "🔪 Nhập thủ công"
            },
            onBackPressed = { navController.popBackStack() }
        )
    }
) { paddingValues ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(paddingValues)
            .verticalScroll(rememberScrollState())
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // == CARD 1: Thông tin dinh dưỡng ==
        Card(
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(
                containerColor = MaterialTheme.colorScheme.surfaceVariant
            )
        ) {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(20.dp),
                verticalArrangement = Arrangement.spacedBy(16.dp)
            ) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(
                        Icons.Filled.Fastfood,
                        contentDescription = null,
                        tint = MaterialTheme.colorScheme.primary
                    )
                    Spacer(modifier = Modifier.width(8.dp))
                    Text(
                        text = "Thông tin dinh dưỡng",
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.Bold
                    )
                }
            }
        }

        // Tên thực phẩm
        OutlinedTextField(
            value = uiState.name,
            onChange = { viewModel.updateName(it) },
            label = { Text("Tên thực phẩm") },
            placeholder = { Text("VD: Ưc gà") },
            leadingIcon = { Icon(Icons.Filled.Label, contentDescription = null) },
            singleLine = true,
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(12.dp)
        )
    }
}

```

```

    )

    // Calo và Protein trên cùng 1 hàng
    Row(
      modifier = Modifier.fillMaxWidth(),
      horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
      OutlinedTextField(
        value = uiState.caloriesPer100g,
        onChange = { viewModel.updateCalories(it) },
        label = { Text("Calo/100g") },
        suffix = { Text("kcal") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        singleLine = true,
        modifier = Modifier.weight(1f),
        shape = RoundedCornerShape(12.dp)
      )
      OutlinedTextField(
        value = uiState.proteinPer100g,
        onChange = { viewModel.updateProtein(it) },
        label = { Text("Protein/100g") },
        suffix = { Text("g") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        singleLine = true,
        modifier = Modifier.weight(1f),
        shape = RoundedCornerShape(12.dp)
      )
    }
  }
}

```

```

// == CARD 2: Thông tin kho ==
Card(
  modifier = Modifier.fillMaxWidth(),
  shape = RoundedCornerShape(16.dp),
  colors = CardDefaults.cardColors(
    containerColor = MaterialTheme.colorScheme.surfaceVariant
  )
) {
  Column(
    modifier = Modifier
      .fillMaxWidth()
      .padding(20.dp),
    verticalArrangement = Arrangement.spacedBy(16.dp)
  ) {
    Row(verticalAlignment = Alignment.CenterVertically) {
      Icon(
        Icons.Filled.Inventory2,
        contentDescription = null,
        tint = MaterialTheme.colorScheme.primary
      )
      Spacer(modifier = Modifier.width(8.dp))
      Text(
        text = "Thông tin bổ sung",
        style = MaterialTheme.typography.titleMedium,
        fontWeight = FontWeight.Bold
      )
    }
  }
}

```

```

// Số lượng và Đơn vị
Row(
  modifier = Modifier.fillMaxWidth(),

```



```

        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        OutlinedTextField(
            value = uiState.quantityGrams,
            onChange = { viewModel.updateQuantity(it) },
            label = { Text("Số lượng") },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            singleLine = true,
            modifier = Modifier.weight(1f),
            shape = RoundedCornerShape(12.dp)
        )

        // Unit dropdown
        ExposedDropdownMenuBox(
            expanded = showUnitDropdown,
            onExpandedChange = { showUnitDropdown = it },
            modifier = Modifier.weight(1f)
        ) {
            OutlinedTextField(
                value = when (uiState.unit) {
                    "gram" -> "Gram (g)"
                    "piece" -> "Cái/Quả"
                    "ml" -> "Mililit (ml)"
                    else -> uiState.unit
                },
                onChange = {},
                readOnly = true,
                label = { Text("Đơn vị") },
                trailingIcon = {
                    ExposedDropdownMenuDefaults.TrailingIcon(expanded = showUnitDropdown)
                },
                modifier = Modifier.menuAnchor(),
                shape = RoundedCornerShape(12.dp)
            )
            ExposedDropdownMenu(
                expanded = showUnitDropdown,
                onDismissRequest = { showUnitDropdown = false }
            ) {
                listOf("gram" to "Gram (g)", "piece" to "Cái/Quả", "ml" to "Mililit (ml)")
                    .forEach { (value, label) ->
                        DropdownMenuItem(
                            text = { Text(label) },
                            onClick = {
                                viewModel.updateUnit(value)
                                showUnitDropdown = false
                            }
                        )
                    }
            }
        }
    }
}

// Ngày hết hạn
OutlinedTextField(
    value = uiState.expiryDateMillis?.let { millis ->
        Instant.ofEpochMilli(millis)
            .atZone(ZoneOffset.UTC)
            .toLocalDate()
            .format(DateTimeFormatter.ofPattern("dd/MM/yyyy"))
    } ?: "Chọn ngày",
    onChange = {},
    readOnly = true,

```

```

        label = { Text("Hạn sử dụng") },
        leadingIcon = { Icon(Icons.Filled.CalendarMonth, contentDescription = null) },
        trailingIcon = {
            IconButton(onClick = { showDatePicker = true }) {
                Icon(Icons.Filled.EditCalendar, contentDescription = "Chọn ngày")
            }
        },
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(12.dp)
    )
}
}

```

```

// == ERROR MESSAGE ==
if (uiState.errorMessage != null) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.errorContainer
        ),
        shape = RoundedCornerShape(12.dp)
    ) {
        Text(
            text = "⚠️ ${uiState.errorMessage}",
            modifier = Modifier.padding(16.dp),
            color = MaterialTheme.colorScheme.onErrorContainer,
            style = MaterialTheme.typography.bodyMedium
        )
    }
}

```

```

// == SAVE BUTTON ==
GradientButton(
    text = if (uiState.isSaving) "Đang lưu..." else "💾 Lưu vào kho",
    onClick = { viewModel.saveToKho() },
    icon = Icons.Filled.Save
)

```

```

    Spacer(modifier = Modifier.height(16.dp))
}
}

```

```

// == DATE PICKER DIALOG ==
if (showDatePicker) {
    val datePickerState = rememberDatePickerState(
        initialSelectedDateMillis = uiState.expiryDateMillis
    )
    DatePickerDialog(
        onDismissRequest = { showDatePicker = false },
        confirmButton = {
            TextButton(
                onClick = {
                    datePickerState.selectedDateMillis?.let { millis ->
                        viewModel.setExpiryDate(millis)
                    }
                }
            )
            showDatePicker = false
        }
    ) { Text("OK") }
},
dismissButton = {
    TextButton(onClick = { showDatePicker = false }) { Text("Hủy") }
}
}

```

```
    ) {  
        DatePicker(state = datePickerState)  
    }  
}  
}
```

File: pantry/PantryListScreen.kt

```

package com.team.smartnutrition.pantry

import androidx.compose.animation.animateColorAsState
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material.icons.outlined.Search
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.team.smartnutrition.R
import com.team.smartnutrition.common.components.ErrorCard
import com.team.smartnutrition.common.components.LoadingScreen
import com.team.smartnutrition.navigation.Screen
import com.team.smartnutrition.pantry.model.PantryItem
import com.team.smartnutrition.pantry.util.ExpiryStatus
import com.team.smartnutrition.pantry.util.calculateExpiryStatus
import com.team.smartnutrition.pantry.util.daysUntilExpiry
import com.team.smartnutrition.pantry.viewmodel.ExpiryFilter
import com.team.smartnutrition.pantry.viewmodel.PantryListViewModel

/**
 * =====
 * MODULE 2 - TV2: DANH SÁCH KHO THỰC PHẨM
 * =====
 *
 * Màn hình chính Module 2:
 * - LazyColumn hiển thị thực phẩm với badge hạn sử dụng (Xanh/Vàng/Đỏ)
 * - Bộ lọc: Tất cả / Sắp hết hạn / Đã hết hạn
 * - Tìm kiếm theo tên
 * - FAB → BottomSheet (Chụp ảnh | Quét barcode | Nhập thủ công)
 * - Swipe để xóa (confirm dialog)
 * - Click vào item → FoodDetailScreen
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PantryListScreen(
    navController: NavController,
    viewModel: PantryListViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()

    Scaffold(
        floatingActionButton = {
            FloatingActionButton(
                onClick = { viewModel.toggleAddSheet(true) },
                containerColor = MaterialTheme.colorScheme.primary,
                contentColor = MaterialTheme.colorScheme.onPrimary
            )
        }
    )

```

```

    ) {
        Icon(Icons.Filled.Add, contentDescription = stringResource(R.string.add_food_btn))
    }
}
) { paddingValues ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(paddingValues)
    ) {
        // === HEADER ===
        Text(
            text = "🍽️ " + stringResource(R.string.pantry_toolbar_title),
            style = MaterialTheme.typography.headlineMedium,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(horizontal = 16.dp, vertical = 12.dp)
        )

        // === SEARCH BAR ===
        OutlinedTextField(
            value = uiState.searchQuery,
            onChange = { viewModel.setSearchQuery(it) },
            placeholder = { Text(stringResource(R.string.search_placeholder)) },
            leadingIcon = {
                Icon(Icons.Outlined.Search, contentDescription = stringResource(R.string.search_placeholder))
            },
            trailingIcon = {
                if (uiState.searchQuery.isNotBlank()) {
                    IconButton(onClick = { viewModel.setSearchQuery("") }) {
                        Icon(Icons.Filled.Clear, contentDescription = stringResource(R.string.delete))
                    }
                }
            },
            singleLine = true,
            shape = RoundedCornerShape(16.dp),
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp)
        )

        Spacer(modifier = Modifier.height(12.dp))

        // === FILTER CHIPS ===
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp),
            horizontalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            ExpiryFilter.entries.forEach { filter ->
                val filterLabel = when (filter) {
                    ExpiryFilter.ALL -> stringResource(R.string.filter_all)
                    ExpiryFilter.EXPIRING -> stringResource(R.string.expiry_status_expiring)
                    ExpiryFilter.EXPIRED -> stringResource(R.string.expiry_status_expired)
                }
                FilterChip(
                    selected = uiState.selectedFilter == filter,
                    onClick = { viewModel.setFilter(filter) },
                    label = { Text(filterLabel) },
                    leadingIcon = if (uiState.selectedFilter == filter) {
                        { Icon(Icons.Filled.Check, contentDescription = null, Modifier.size(18.dp)) }
                    } else null
                )
            }
        }
    }
}

```



```

    },
    onBarcodeClick = {
        viewModel.toggleAddSheet(false)
        navController.navigate(Screen.BarcodeScan.route)
    },
    onManualClick = {
        viewModel.toggleAddSheet(false)
        // Navigate FoodResult với source>manual (empty form)
        navController.currentBackStackEntry?.savedStateHandle?.set("food_source", "manual")
        navController.navigate(Screen.FoodResult.route)
    }
}
)
}
}

// == DELETE CONFIRM DIALOG ==
if (uiState.showDeleteDialog && uiState.itemToDelete != null) {
    AlertDialog(
        onDismissRequest = { viewModel.dismissDeleteConfirm() },
        title = { Text(stringResource(R.string.delete_food_title)) },
        text = {
            Text(stringResource(R.string.delete_food_confirm, uiState.itemToDelete!!.name))
        },
        confirmButton = {
            TextButton(
                onClick = { viewModel.deleteItem(uiState.itemToDelete!!) },
                colors = ButtonDefaults.textButtonColors(
                    contentColor = MaterialTheme.colorScheme.error
                )
            ) { Text(stringResource(R.string.delete)) }
        },
        dismissButton = {
            TextButton(onClick = { viewModel.dismissDeleteConfirm() }) { Text(stringResource(R.string.cancel)) }
        }
    )
}

// =====
// SUB-COMPOSABLES
// =====

/**
 * Item thực phẩm với swipe-to-delete.
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
private fun SwipeToDismissItem(
    item: PantryItem,
    onDelete: () -> Unit,
    onClick: () -> Unit
) {
    val dismissState = rememberSwipeToDismissBoxState(
        confirmValueChange = { dismissValue ->
            if (dismissValue == SwipeToDismissBoxValue.EndToStart) {
                onDelete()
                false // Không tự dismiss, chờ confirm dialog
            } else false
        }
    )
}

SwipeToDismissBox(

```

```

        state = dismissState,
        backgroundColor = {
            // Background đỏ khi swipe
            Box(
                modifier = Modifier
                    .fillMaxSize()
                    .background(
                        MaterialTheme.colorScheme.errorContainer,
                        RoundedCornerShape(16.dp)
                    )
                    .padding(horizontal = 20.dp),
                contentAlignment = Alignment.CenterEnd
            ) {
                Icon(
                    Icons.Filled.Delete,
                    contentDescription = stringResource(R.string.delete),
                    tint = MaterialTheme.colorScheme.onErrorContainer
                )
            }
        },
        enableDismissFromStartToEnd = false
    ) {
        PantryItemCard(item = item, onClick = onClick)
    }
}

/**
 * Card hiển thị 1 thực phẩm.
 */
@Composable
private fun PantryItemCard(
    item: PantryItem,
    onClick: () -> Unit
) {
    val expiryStatus = calculateExpiryStatus(item.expiryDate)
    val days = daysUntilExpiry(item.expiryDate)

    Card(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface
        ),
        elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Icon theo nguồn
            Surface(
                modifier = Modifier.size(48.dp),
                shape = RoundedCornerShape(12.dp),
                color = MaterialTheme.colorScheme.primaryContainer
            ) {
                Box(contentAlignment = Alignment.Center) {
                    Icon(
                        imageVector = when (item.source) {

```



```

        "camera" -> Icons.Filled.CameraAlt
        "barcode" -> Icons.Filled.QrCodeScanner
        else -> Icons.Filled.Edit
    },
    contentDescription = null,
    tint = MaterialTheme.colorScheme.onPrimaryContainer,
    modifier = Modifier.size(24.dp)
)
}
}

Spacer(modifier = Modifier.width(12.dp))

// Thông tin
Column(modifier = Modifier.weight(1f)) {
    Text(
        text = item.name,
        style = MaterialTheme.typography.titleMedium,
        fontWeight = FontWeight.SemiBold,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis
    )
    Spacer(modifier = Modifier.height(2.dp))
    Text(
        text = "${item.caloriesPer100g} kcal • ${item.proteinPer100g}g protein • ${item.quantityGrams}${item.unit}",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        maxLines = 1,
        overflow = TextOverflow.Ellipsis
    )
}

Spacer(modifier = Modifier.width(8.dp))

// Status Badge
StatusBadge(status = expiryStatus, daysLeft = days)
}
}
}

/**
 * Badge hiển thị trạng thái hạn sử dụng (Xanh/Vàng/Đỏ).
 */
@Composable
fun StatusBadge(
    status: ExpiryStatus,
    daysLeft: Long,
    modifier: Modifier = Modifier
) {
    val backgroundColor = Color(android.graphics.Color.parseColor(status.colorHex))

    Surface(
        modifier = modifier,
        shape = RoundedCornerShape(12.dp),
        color = backgroundColor
    ) {
        Text(
            text = when (status) {
                ExpiryStatus.FRESH -> "${daysLeft}d"
                ExpiryStatus.EXPIRING -> "${daysLeft}d ⚠️"
                ExpiryStatus.EXPIRED -> stringResource(R.string.expiry_status_expired)
            },

```

```

        style = MaterialTheme.typography.labelSmall,
        color = Color.White,
        fontWeight = FontWeight.Bold,
        modifier = Modifier.padding(horizontal = 8.dp, vertical = 4.dp)
    )
}

/**
 * Empty state khi kho trống.
 */
@Composable
private fun EmptyPantryState(isFiltered: Boolean) {
    Box(
        modifier = Modifier
            .fillMaxSize()
            .padding(32.dp),
        contentAlignment = Alignment.Center
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Icon(
                imageVector = if (isFiltered) Icons.Filled.SearchOff else Icons.Filled.Kitchen,
                contentDescription = null,
                modifier = Modifier.size(80.dp),
                tint = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.5f)
            )
            Spacer(modifier = Modifier.height(16.dp))
            Text(
                text = if (isFiltered) stringResource(R.string.no_food_found)
                else stringResource(R.string.pantry_empty),
                style = MaterialTheme.typography.titleLarge,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = if (isFiltered) stringResource(R.string.filter_or_search_hint)
                else stringResource(R.string.pantry_empty_hint),
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.7f),
                textAlign = TextAlign.Center
            )
        }
    }
}

/**
 * BottomSheet chọn cách thêm thực phẩm.
 */
@Composable
private fun AddFoodBottomSheet(
    onCameraClick: () -> Unit,
    onBarcodeClick: () -> Unit,
    onManualClick: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(24.dp)
    ) {
        Text(
            text = stringResource(R.string.add_food_btn),
            style = MaterialTheme.typography.titleLarge,

```

```

        fontWeight = FontWeight.Bold
    )
    Spacer(modifier = Modifier.height(20.dp))

    // Option 1: Camera AI
    AddOptionItem(
        icon = Icons.Filled.CameraAlt,
        title = stringResource(R.string.photo_ai_title),
        description = stringResource(R.string.photo_ai_desc),
        onClick = onCameraClick
    )
    Spacer(modifier = Modifier.height(12.dp))

    // Option 2: Barcode
    AddOptionItem(
        icon = Icons.Filled.QrCodeScanner,
        title = stringResource(R.string.barcode_scan_title),
        description = stringResource(R.string.barcode_scan_desc),
        onClick = onBarcodeClick
    )
    Spacer(modifier = Modifier.height(12.dp))

    // Option 3: Manual
    AddOptionItem(
        icon = Icons.Filled.Edit,
        title = stringResource(R.string.manual_entry_title),
        description = stringResource(R.string.manual_entry_desc),
        onClick = onManualClick
    )

    Spacer(modifier = Modifier.height(24.dp))
}

/**
 * Item trong BottomSheet thêm thực phẩm.
 */
@Composable
private fun AddOptionItem(
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    title: String,
    description: String,
    onClick: () -> Unit
) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant
        )
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Surface(
                modifier = Modifier.size(48.dp),
                shape = RoundedCornerShape(12.dp),

```

```
        color = MaterialTheme.colorScheme.primary.copy(alpha = 0.1f)
    ) {
        Box(contentAlignment = Alignment.Center) {
            Icon(
                imageVector = icon,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.primary,
                modifier = Modifier.size(24.dp)
            )
        }
    }
    Spacer(modifier = Modifier.width(16.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            text = title,
            style = MaterialTheme.typography.titleSmall,
            fontWeight = FontWeight.SemiBold
        )
        Text(
            text = description,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
    Icon(
        Icons.Filled.ChevronRight,
        contentDescription = null,
        tint = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}
```

File: pantry/data/GeminiService.kt

```

package com.team.smartnutrition.pantry.data

import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.util.Log
import com.google.ai.client.generativeai.GenerativeModel
import com.google.ai.client.generativeai.type.content
import com.google.gson.Gson
import com.google.gson.JsonSyntaxException
import com.team.smartnutrition.BuildConfig
import com.team.smartnutrition.pantry.model.FoodRecognitionResult
import java.io.ByteArrayOutputStream

/**
 * =====
 * GEMINI SERVICE - Gọi Gemini Vision API
 * =====
 *
 * Nhận diện thực phẩm từ ảnh chụp bằng Gemini Vision API.
 * Pipeline: Bitmap → Resize → Gemini API → JSON → FoodRecognitionResult
 *
 * Lưu ý:
 * - API Key lấy từ BuildConfig.GEMINI_API_KEY (local.properties)
 * - Ảnh resize xuống max 1024×1024 để tránh request quá 4MB
 * - Gemini đôi khi trả markdown wrapper → cần strip trước khi parse JSON
 */
class GeminiService {

    companion object {
        private const val TAG = "GeminiService"
        private const val MAX_IMAGE_SIZE = 1024
        private const val MODEL_NAME = "gemini-2.5-flash"

        /** Prompt nhận diện thực phẩm (tiếng Việt) */
        private const val FOOD_RECOGNITION_PROMPT = """
Bạn là chuyên gia dinh dưỡng. Nhận diện thực phẩm trong ảnh này.
Trả về ĐÚNG 1 JSON object với format sau:
{"name": "tên tiếng Việt", "calories": số kcal trên 100g, "protein": số gram protein trên 100g}
        """
    }

    Quy tắc:
    - "name": tên thực phẩm bằng tiếng Việt, viết thường, ngắn gọn
    - "calories": số nguyên, ước tính kcal trên 100g
    - "protein": số nguyên, ước tính gram protein trên 100g
    - Chỉ trả JSON thuần, KHÔNG giải thích thêm, KHÔNG wrap trong markdown
    """
    }

    private val model: GenerativeModel by lazy {
        val apiKey = BuildConfig.GEMINI_API_KEY
        if (apiKey.isBlank()) {
            throw IllegalStateException(
                "GEMINI_API_KEY chưa được cấu hình. " +
                "Thêm dòng GEMINI_API_KEY=your_key vào file local.properties"
            )
        }
        GenerativeModel(
            modelName = MODEL_NAME,
            apiKey = apiKey
        )
    }

    /**

```

```

* Nhận diện thực phẩm từ ảnh Bitmap.
*
* @param bitmap ảnh chụp thực phẩm (sẽ tự resize nếu quá lớn)
* @return FoodRecognitionResult chứa tên, calo, protein
* @throws Exception nếu API call hoặc JSON parse thất bại
*/
suspend fun recognizeFood(bitmap: Bitmap): FoodRecognitionResult {
    // 1. Resize bitmap nếu cần
    val resizedBitmap = resizeBitmap(bitmap)

    // 2. Gọi Gemini Vision API
    val response = kotlinx.coroutines.withTimeoutOrNull(30000) {
        model.generateContent(
            content {
                image(resizedBitmap)
                text(FOOD_RECOGNITION_PROMPT)
            }
        )
    } ?: throw Exception("Timeout khi gọi AI nhận diện thực phẩm")

    val responseText = response.text
    ?: throw Exception("AI không trả về kết quả")

    Log.d(TAG, "Gemini response: $responseText")

    // 3. Parse JSON (strip markdown wrapper nếu có)
    return parseResponse(responseText)
}

/**
 * Resize bitmap xuống max 1024x1024, giữ tỷ lệ.
 * Nếu bitmap đã nhỏ hơn → trả nguyên.
 */
private fun resizeBitmap(bitmap: Bitmap): Bitmap {
    val width = bitmap.width
    val height = bitmap.height

    if (width <= MAX_IMAGE_SIZE && height <= MAX_IMAGE_SIZE) {
        return bitmap
    }

    val ratio = minOf(
        MAX_IMAGE_SIZE.toFloat() / width,
        MAX_IMAGE_SIZE.toFloat() / height
    )
    val newWidth = (width * ratio).toInt()
    val newHeight = (height * ratio).toInt()

    return Bitmap.createScaledBitmap(bitmap, newWidth, newHeight, true)
}

/**
 * Parse response text từ Gemini thành FoodRecognitionResult.
 * Xử lý các trường hợp Gemini wrap JSON trong markdown code block.
 */
private fun parseResponse(responseText: String): FoodRecognitionResult {
    // Strip markdown wrapper nếu có
    val jsonString = stripMarkdownWrapper(responseText)

    return try {
        val result = Gson().fromJson(jsonString, FoodRecognitionResult::class.java)
        // Validate

```

```
        if (result.name.isBlank()) {
            throw Exception("AI không nhận diện được tên thực phẩm")
        }
        result
    } catch (e: JsonSyntaxException) {
        Log.e(TAG, "JSON parse error: $jsonString", e)
        throw Exception("Không thể đọc kết quả từ AI. Vui lòng chụp lại ảnh rõ hơn.")
    }
}

/**
 * Strip markdown code block wrapper.
 * Gemini đôi khi trả: ```json\n{...}\n``` thay vì pure JSON.
 */
private fun stripMarkdownWrapper(text: String): String {
    var result = text.trim()

    // Remove ```json ... ``` wrapper
    if (result.startsWith("```")) {
        result = result.removePrefix("```json")
            .removePrefix("```JSON")
            .removePrefix("```")
        if (result.endsWith("```")) {
            result = result.removeSuffix("```")
        }
    }

    return result.trim()
}
}
```

File: pantry/data/PantryRepository.kt

```

package com.team.smartnutrition.pantry.data

import com.google.firebase.Timestamp
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.Query
import com.team.smartnutrition.pantry.model.PantryItem
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.tasks.await

/**
 * =====
 * PANTRY REPOSITORY - Lớp dữ liệu kho thực phẩm
 * =====
 *
 * Đóng gói toàn bộ Firestore operations cho collection:
 * users/{uid}/pantry/{autoId}
 *
 * Convention: giống UserRepository (Module 1)
 * - Dùng withTimeoutOrNull cho mọi Firestore call
 * - Suspend functions cho one-shot operations
 * - Flow cho realtime listeners
 */
class PantryRepository {

    private val firestore = FirebaseFirestore.getInstance()

    /** UID của user hiện tại */
    val currentUid: String?
        get() = FirebaseAuth.getInstance().currentUser?.uid

    // == HELPER ==

    /** Reference tới pantry collection của user */
    private fun pantryRef(uid: String) =
        firestore.collection("users").document(uid).collection("pantry")

    // == CREATE ==

    /**
     * Thêm thực phẩm mới vào kho.
     * @return document ID được tạo tự động
     */
    suspend fun addItem(uid: String, item: PantryItem): String {
        val data = itemToMap(item).toMutableMap()
        data["addedAt"] = Timestamp.now()

        // Tạo document ID tự động trước ở client
        val docRef = pantryRef(uid).document()
        // Ghi bất đồng bộ (offline-first) - ghi lập tức vào SQLite cache của thiết bị
        docRef.set(data)

        return docRef.id
    }

    // == READ ==

    /**
     * Lấy realtime danh sách tất cả thực phẩm trong kho.
     * Sắp xếp theo ngày hết hạn (sắp hết hạn lên trước).

```



```

    * @return Flow tự cập nhật khi data thay đổi
    */
    fun getItem(uid: String): Flow<List<PantryItem>> = callbackFlow {
        val listener = pantryRef(uid)
            .orderBy("expiryDate", Query.Direction.ASCENDING)
            .addSnapshotListener { snapshot, error ->
                if (error != null) {
                    close(error)
                    return@addSnapshotListener
                }
                val items = snapshot?.documents?.map { doc ->
                    doc.toObject(PantryItem::class.java)?.copy(id = doc.id)
                }?: emptyList()
                trySend(items)
            }
        awaitClose { listener.remove() }
    }

    /**
     * Lấy 1 thực phẩm theo document ID.
     * @return PantryItem hoặc null nếu không tồn tại
     */
    suspend fun getItem(uid: String, itemId: String): PantryItem? {
        val doc = try {
            kotlinx.coroutines.withTimeout(3000) {
                pantryRef(uid).document(itemId).get().await()
            }
        } catch (e: Exception) {
            // Fallback đọc từ cache local
            pantryRef(uid).document(itemId).get(com.google.firebase.firestore.Source.CACHE).await()
        }

        return doc.toObject(PantryItem::class.java)?.copy(id = doc.id)
    }

    /**
     * Lấy danh sách thực phẩm chưa hết hạn, sắp xếp sắp hết hạn trước.
     * Dùng cho Module 3 (AI Meal Planner) để lấy nguyên liệu.
     */
    suspend fun getAvailableItems(uid: String): List<PantryItem> {
        val snapshot = try {
            kotlinx.coroutines.withTimeout(3000) {
                pantryRef(uid)
                    .whereNotEqualTo("status", "expired")
                    .orderBy("status")
                    .orderBy("expiryDate", Query.Direction.ASCENDING)
                    .get()
                    .await()
            }
        } catch (e: Exception) {
            // Fallback đọc từ cache local
            pantryRef(uid)
                .whereNotEqualTo("status", "expired")
                .orderBy("status")
                .orderBy("expiryDate", Query.Direction.ASCENDING)
                .get(com.google.firebase.firestore.Source.CACHE)
                .await()
        }

        return snapshot.documents.map { doc ->
            doc.toObject(PantryItem::class.java)?.copy(id = doc.id)
        }
    }

```

```

        ?: PantryItem(id = doc.id)
    }
}

// == UPDATE ==

/**
 * Cập nhật fields cụ thể của 1 thực phẩm.
 * @param updates Map chứa field name → value mới
 */
suspend fun updateItem(uid: String, itemId: String, updates: Map<String, Any>) {
    // Ghi bất đồng bộ (offline-first)
    pantryRef(uid).document(itemId).update(updates)
}

// == DELETE ==

/**
 * Xóa 1 thực phẩm khỏi kho.
 */
suspend fun deleteItem(uid: String, itemId: String) {
    // Ghi bất đồng bộ (offline-first)
    pantryRef(uid).document(itemId).delete()
}

// == HELPER: Convert PantryItem → Map ==

/**
 * Convert PantryItem thành Map để lưu Firestore.
 * Không bao gồm 'id' vì đó là document ID.
 */
private fun itemToMap(item: PantryItem): Map<String, Any?> {
    return mapOf(
        "name" to item.name,
        "caloriesPer100g" to item.caloriesPer100g,
        "proteinPer100g" to item.proteinPer100g,
        "quantityGrams" to item.quantityGrams,
        "unit" to item.unit,
        "source" to item.source,
        "imageUrl" to item.imageUrl,
        "expiryDate" to item.expiryDate,
        "status" to item.status,
        "barcode" to item.barcode
    )
}
}

```

File: pantry/model/PantryItem.kt

```
package com.team.smartnutrition.pantry.model

import com.google.firebase.Timestamp

/**
 * =====
 * DATA MODELS cho Module 2 - Pantry Scanner
 * =====
 *
 * Ảnh xạ trực tiếp tới Firestore document:
 * - PantryItem → users/{uid}/pantry/{autoId}
 * - FoodRecognitionResult → kết quả từ Gemini Vision API
 */

/**
 * Thực phẩm trong kho, khớp 1:1 với Firestore schema.
 * Dùng default values để Firestore.toObject() hoạt động.
 */
data class PantryItem(
    val id: String = "", // Firestore document ID
    val name: String = "", // Tên thực phẩm (VD: "Ức gà")
    val caloriesPer100g: Int = 0, // kcal trên 100g
    val proteinPer100g: Int = 0, // gram protein trên 100g
    val quantityGrams: Int = 0, // Tổng gram đang có
    val unit: String = "gram", // "gram" | "piece" | "ml"
    val source: String = "camera", // "camera" | "barcode" | "manual"
    val imageUrl: String = "", // (optional) URL ảnh đã chụp
    val expiryDate: Timestamp? = null, // Ngày hết hạn
    val addedAt: Timestamp? = null, // Ngày thêm vào kho
    val status: String = "fresh", // "fresh" | "expiring" | "expired"
    val barcode: String = "" // Mã vạch nếu có
)

/**
 * Kết quả nhận diện thực phẩm từ Gemini Vision API.
 * Dùng để truyền giữa CameraCapture/BarcodeScan → FoodResult.
 */
data class FoodRecognitionResult(
    val name: String = "",
    val calories: Int = 0, // kcal trên 100g
    val protein: Int = 0 // gram protein trên 100g
)
```

File: pantry/util/BarcodeDatabase.kt

```

package com.team.smartnutrition.pantry.util

import com.team.smartnutrition.pantry.model.FoodRecognitionResult

/**
 * =====
 * BARCODE DATABASE - Tra cứu sản phẩm VN local
 * =====
 *
 * HashMap hardcoded các sản phẩm phổ biến tại Việt Nam.
 * Dùng làm fallback khi không có API tra cứu online.
 *
 * Barcode format: EAN-13 (13 số) – chuẩn phổ biến nhất tại VN.
 * Prefix 893 = Vietnam.
 */
object BarcodeDatabase {

    private val products = mapOf(
        // === MÌ GÓI ===
        "8934563138028" to FoodRecognitionResult("Mì Hào Hào tôm chua cay", 360, 8),
        "8934563138011" to FoodRecognitionResult("Mì Hào Hào sa tế hành", 355, 7),
        "8936136160019" to FoodRecognitionResult("Mì Omachi sốt bò hầm", 370, 9),
        "8934680025256" to FoodRecognitionResult("Mì Kokomi tôm chua cay", 350, 7),

        // === SỮA ===
        "8934673583220" to FoodRecognitionResult("Sữa Vinamilk có đường", 67, 3),
        "8934673583237" to FoodRecognitionResult("Sữa Vinamilk không đường", 46, 3),
        "8936036020427" to FoodRecognitionResult("Sữa TH True Milk", 63, 3),
        "8934804019529" to FoodRecognitionResult("Sữa đậu nành Fami", 42, 3),
        "8934673573344" to FoodRecognitionResult("Sữa tươi Vinamilk có đường 180ml", 76, 3),

        // === NƯỚC GIẢI KHÁT ===
        "8935049500100" to FoodRecognitionResult("Trà xanh Không Độ", 18, 0),
        "8934588013058" to FoodRecognitionResult("Nước tăng lực Sting", 49, 0),
        "8935049501107" to FoodRecognitionResult("Nước C2 trà chanh", 30, 0),
        "5449000000996" to FoodRecognitionResult("Coca-Cola", 42, 0),
        "8934588062001" to FoodRecognitionResult("Pepsi", 44, 0),
        "8938535231014" to FoodRecognitionResult("Nước yến nha đam 500ml", 40, 0),

        // === BÁNH KẸO ===
        "8934680027205" to FoodRecognitionResult("Bánh Chocopie", 440, 5),
        "8934680010016" to FoodRecognitionResult("Bánh Orion Custas", 400, 6),

        // === GIA VỊ ===
        "8934804032160" to FoodRecognitionResult("Nước mắm Chinsu", 50, 8),
        "8936017360019" to FoodRecognitionResult("Nước tương Maggi", 60, 10),
        "8934563960018" to FoodRecognitionResult("Hạt nêm Knorr", 210, 12),

        // === THỰC PHẨM ĐÓNG GÓI ===
        "8938506556015" to FoodRecognitionResult("Cá hộp Ba Cô Gái", 200, 25),
        "8934563789015" to FoodRecognitionResult("Xúc xích Vissan", 280, 12),
    )

    /**
     * Tra cứu sản phẩm theo mã vạch.
     * @param barcode chuỗi số barcode (EAN-13)
     * @return FoodRecognitionResult nếu tìm thấy, null nếu không có trong database
     */
    fun lookup(barcode: String): FoodRecognitionResult? {
        return products[barcode.trim()]
    }
}

```

```
/**
 * Kiểm tra barcode có trong database không.
 */
fun contains(barcode: String): Boolean {
    return products.containsKey(barcode.trim())
}

/**
 * Tổng số sản phẩm trong database.
 */
val size: Int get() = products.size
}
```

File: pantry/util/ExpiryStatus.kt

```

package com.team.smartnutrition.pantry.util

import com.google.firebase.Timestamp
import java.time.LocalDate
import java.time.ZoneId
import java.time.temporal.ChronoUnit

/**
 * =====
 * EXPIRY STATUS - Logic cảnh báo hạn sử dụng
 * =====
 *
 * Quy tắc:
 * - 🟢 FRESH: còn > 3 ngày
 * - 🟡 EXPIRING: còn 1-3 ngày
 * - 🔴 EXPIRED: đã hết hạn (≤ 0 ngày)
 */
enum class ExpiryStatus(
    val label: String,
    val colorHex: String,
    val firestoreValue: String
) {
    FRESH("Còn tươi", "#4CAF50", "fresh"),
    EXPIRING("Sắp hết hạn!", "#FFC107", "expiring"),
    EXPIRED("ĐÃ HẾT HẠN", "#F44336", "expired");
}

/**
 * Tính số ngày còn lại cho đến khi hết hạn.
 * @return số ngày (âm nếu đã quá hạn), hoặc Long.MAX_VALUE nếu không có ngày hết hạn
 */
fun daysUntilExpiry(expiryDate: Timestamp?): Long {
    if (expiryDate == null) return Long.MAX_VALUE

    val expiryLocalDate = expiryDate.toDate().toInstant()
        .atZone(ZoneId.systemDefault())
        .toLocalDate()

    return ChronoUnit.DAYS.between(LocalDate.now(), expiryLocalDate)
}

/**
 * Xác định trạng thái hạn sử dụng dựa trên ngày hết hạn.
 * @return ExpiryStatus tương ứng
 */
fun calculateExpiryStatus(expiryDate: Timestamp?): ExpiryStatus {
    val days = daysUntilExpiry(expiryDate)

    return when {
        days > 3 -> ExpiryStatus.FRESH
        days >= 1 -> ExpiryStatus.EXPIRING
        else -> ExpiryStatus.EXPIRED
    }
}

/**
 * Tạo text hiển thị cho trạng thái hạn sử dụng.
 * VD: "Còn 5 ngày", "Sắp hết hạn!", "ĐÃ HẾT HẠN"
 */
fun expiryDisplayText(expiryDate: Timestamp?): String {
    val days = daysUntilExpiry(expiryDate)
    val status = calculateExpiryStatus(expiryDate)

```

```
return when (status) {  
    ExpiryStatus.FRESH -> "Còn $days ngày"  
    ExpiryStatus.EXPIRING -> "Còn $days ngày - Sắp hết hạn!"  
    ExpiryStatus.EXPIRED -> {  
        if (days == 0L) "Hết hạn hôm nay"  
        else "Đã hết hạn ${-days} ngày"  
    }  
}  
}
```

File: pantry/viewmodel/BarcodeScanViewModel.kt

```

package com.team.smartnutrition.pantry.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.gson.Gson
import com.team.smartnutrition.pantry.model.FoodRecognitionResult
import com.team.smartnutrition.pantry.util.BarcodeDatabase
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * BARCODE SCAN VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Trạng thái quyền camera
 * - Phát hiện barcode từ ML Kit
 * - Tra cứu sản phẩm trong BarcodeDatabase local
 * - Trả kết quả để navigate sang FoodResult
 */

data class BarcodeScanUiState(
    val hasCameraPermission: Boolean = false,
    val detectedBarcode: String? = null,
    val lookupResult: FoodRecognitionResult? = null,
    val isLookingUp: Boolean = false,
    val notFound: Boolean = false,
    val navigateToResult: Boolean = false,
    val resultJson: String? = null,
    val showResultSheet: Boolean = false,
    val showNotFoundDialog: Boolean = false,
    val errorMessage: String? = null
)

class BarcodeScanViewModel : ViewModel() {

    private val gson = Gson()
    private var processedBarcode: String? = null // Tránh xử lý trùng

    private val _uiState = MutableStateFlow(BarcodeScanUiState())
    val uiState: StateFlow<BarcodeScanUiState> = _uiState.asStateFlow()

    /**
     * Cập nhật trạng thái quyền camera.
     */
    fun updatePermission(granted: Boolean) {
        _uiState.update { it.copy(hasCameraPermission = granted) }
    }

    /**
     * Xử lý khi ML Kit phát hiện barcode.
     * Chỉ xử lý 1 lần cho mỗi mã (tránh trigger liên tục).
     */
    fun onBarcodeDetected(rawValue: String) {
        // Tránh xử lý trùng
        if (rawValue == processedBarcode) return
        processedBarcode = rawValue
    }

```



```

        _uiState.update {
            it.copy(
                detectedBarcode = rawValue,
                isLookingUp = true,
                notFound = false,
                lookupResult = null
            )
        }

        viewModelScope.launch {
            val result = BarcodeDatabase.lookup(rawValue)

            if (result != null) {
                _uiState.update {
                    it.copy(
                        lookupResult = result,
                        isLookingUp = false,
                        showResultSheet = true,
                        resultJson = gson.toJson(result)
                    )
                }
            } else {
                _uiState.update {
                    it.copy(
                        notFound = true,
                        isLookingUp = false,
                        showNotFoundDialog = true
                    )
                }
            }
        }
    }

    /**
     * User xác nhận sản phẩm tìm thấy → navigate FoodResult.
     */
    fun confirmResult() {
        _uiState.update {
            it.copy(
                showResultSheet = false,
                navigateToResult = true
            )
        }
    }

    /**
     * Reset để quét barcode mới.
     */
    fun resetScan() {
        processedBarcode = null
        _uiState.update {
            it.copy(
                detectedBarcode = null,
                lookupResult = null,
                notFound = false,
                showResultSheet = false,
                showNotFoundDialog = false,
                navigateToResult = false,
                resultJson = null
            )
        }
    }
}

```

```
/**
 * Navigate FoodResult cho manual entry (barcode không tìm thấy).
 */
fun navigateManualEntry() {
    _uiState.update {
        it.copy(
            showNotFoundDialog = false,
            navigateToResult = true,
            resultJson = null // null = empty form
        )
    }
}

/** Reset navigation state */
fun onNavigated() {
    _uiState.update { it.copy(navigateToResult = false) }
}

/** Dismiss result sheet */
fun dismissResultSheet() {
    _uiState.update { it.copy(showResultSheet = false) }
    resetScan()
}

/** Dismiss not found dialog */
fun dismissNotFoundDialog() {
    _uiState.update { it.copy(showNotFoundDialog = false) }
    resetScan()
}
}
```

File: pantry/viewmodel/CameraCaptureViewModel.kt

```

package com.team.smartnutrition.pantry.viewmodel

import android.graphics.Bitmap
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.gson.Gson
import com.team.smartnutrition.pantry.data.GeminiService
import com.team.smartnutrition.pantry.model.FoodRecognitionResult
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * CAMERA CAPTURE VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Trạng thái quyền camera
 * - Capture ảnh từ CameraX
 * - Gửi ảnh đến Gemini Vision API nhận diện
 * - Trả kết quả dạng JSON string để truyền qua navigation
 */

data class CameraCaptureUiState(
    val hasCameraPermission: Boolean = false,
    val isProcessing: Boolean = false,
    val errorMessage: String? = null,
    val resultJson: String? = null, // JSON string để truyền qua savedStateHandle
    val navigateToResult: Boolean = false
)

class CameraCaptureViewModel : ViewModel() {

    private val geminiService = GeminiService()
    private val gson = Gson()

    private val _uiState = MutableStateFlow(CameraCaptureUiState())
    val uiState: StateFlow<CameraCaptureUiState> = _uiState.asStateFlow()

    /**
     * Cập nhật trạng thái quyền camera.
     */
    fun updatePermission(granted: Boolean) {
        _uiState.update { it.copy(hasCameraPermission = granted) }
    }

    /**
     * Xử lý ảnh đã chụp: gửi đến Gemini Vision API.
     */
    fun onPhotoCaptured(bitmap: Bitmap) {
        viewModelScope.launch {
            _uiState.update { it.copy(isProcessing = true, errorMessage = null) }

            try {
                val result = geminiService.recognizeFood(bitmap)
                val json = gson.toJson(result)

                _uiState.update {
                    it.copy(

```

```
        isProcessing = false,
        resultJson = json,
        navigateToResult = true
    )
}
} catch (e: Exception) {
    _uiState.update {
        it.copy(
            isProcessing = false,
            errorMessage = e.message ?: "Lỗi nhận diện thực phẩm"
        )
    }
}
}
}

/** Reset navigation state sau khi đã navigate */
fun onNavigated() {
    _uiState.update { it.copy(navigateToResult = false, resultJson = null) }
}

/** Clear error */
fun clearError() {
    _uiState.update { it.copy(errorMessage = null) }
}
}
```

File: pantry/viewmodel/FoodDetailViewModel.kt

```

package com.team.smartnutrition.pantry.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.team.smartnutrition.pantry.data.PantryRepository
import com.team.smartnutrition.pantry.model.PantryItem
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * FOOD DETAIL VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Load chi tiết 1 thực phẩm theo itemId
 * - Chỉnh sửa số lượng
 * - Xóa thực phẩm
 */

data class FoodDetailUiState(
    val item: PantryItem? = null,
    val isLoading: Boolean = true,
    val isEditing: Boolean = false,
    val editQuantity: String = "",
    val isSaving: Boolean = false,
    val isDeleting: Boolean = false,
    val deleteSuccess: Boolean = false,
    val saveSuccess: Boolean = false,
    val showDeleteDialog: Boolean = false,
    val errorMessage: String? = null
)

class FoodDetailViewModel : ViewModel() {

    private val repository = PantryRepository()

    private val _uiState = MutableStateFlow(FoodDetailUiState())
    val uiState: StateFlow<FoodDetailUiState> = _uiState.asStateFlow()

    /**
     * Load chi tiết item từ Firestore.
     */
    fun loadItem(itemId: String) {
        val uid = repository.currentUid ?: return

        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            try {
                val item = repository.getItem(uid, itemId)
                if (item != null) {
                    _uiState.update {
                        it.copy(
                            item = item,
                            isLoading = false,
                            editQuantity = item.quantityGrams.toString()
                        )
                    }
                }
            } else {

```

```

        _uiState.update {
            it.copy(isLoading = false, errorMessage = "Thực phẩm không tồn tại")
        }
    }
} catch (e: Exception) {
    _uiState.update {
        it.copy(isLoading = false, errorMessage = "Lỗi: ${e.message}")
    }
}
}
}

// == EDIT MODE ==

fun startEditing() {
    _uiState.update { state ->
        state.copy(
            isEditing = true,
            editQuantity = state.item?.quantityGrams?.toString() ?: "0"
        )
    }
}

fun cancelEditing() {
    _uiState.update { state ->
        state.copy(
            isEditing = false,
            editQuantity = state.item?.quantityGrams?.toString() ?: "0"
        )
    }
}

fun updateEditQuantity(value: String) {
    _uiState.update { it.copy(editQuantity = value) }
}

fun saveChanges() {
    val uid = repository.currentUid ?: return
    val item = _uiState.value.item ?: return
    val newQuantity = _uiState.value.editQuantity.toIntOrNull()

    if (newQuantity == null || newQuantity <= 0) {
        _uiState.update { it.copy(errorMessage = "Số lượng phải lớn hơn 0") }
        return
    }

    viewModelScope.launch {
        _uiState.update { it.copy(isSaving = true, errorMessage = null) }
        try {
            repository.updateItem(
                uid, item.id,
                mapOf("quantityGrams" to newQuantity)
            )
            _uiState.update { state ->
                state.copy(
                    isSaving = false,
                    isEditing = false,
                    item = state.item?.copy(quantityGrams = newQuantity)
                )
            }
        } catch (e: Exception) {
            _uiState.update {

```

```
        it.copy(isSaving = false, errorMessage = "Lưu thất bại: ${e.message}")
    }
}

}

}

}

// == DELETE ==

fun showDeleteConfirm() {
    _uiState.update { it.copy(showDeleteDialog = true) }
}

fun dismissDeleteConfirm() {
    _uiState.update { it.copy(showDeleteDialog = false) }
}

fun deleteItem() {
    val uid = repository.currentUid ?: return
    val item = _uiState.value.item ?: return

    viewModelScope.launch {
        _uiState.update { it.copy(isDeleting = true, showDeleteDialog = false) }
        try {
            repository.deleteItem(uid, item.id)
            _uiState.update { it.copy(isDeleting = false, deleteSuccess = true) }
        } catch (e: Exception) {
            _uiState.update {
                it.copy(isDeleting = false, errorMessage = "Xóa thất bại: ${e.message}")
            }
        }
    }
}

/** Clear error */
fun clearError() {
    _uiState.update { it.copy(errorMessage = null) }
}
}
```

File: pantry/viewmodel/FoodResultViewModel.kt

```

package com.team.smartnutrition.pantry.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.firebase.Timestamp
import com.google.gson.Gson
import com.team.smartnutrition.pantry.data.PantryRepository
import com.team.smartnutrition.pantry.model.FoodRecognitionResult
import com.team.smartnutrition.pantry.model.PantryItem
import com.team.smartnutrition.pantry.util.calculateExpiryStatus
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
import java.time.Instant
import java.time.LocalDate
import java.time.ZoneId
import java.time.ZoneOffset

/**
 * =====
 * FOOD RESULT VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Hiển thị kết quả AI nhận diện (pre-filled)
 * - Cho user chỉnh sửa tên, calo, protein
 * - Chọn số lượng, đơn vị, ngày hết hạn
 * - Lưu vào Firestore pantry collection
 */

data class FoodResultUiState(
    val name: String = "",
    val caloriesPer100g: String = "0",
    val proteinPer100g: String = "0",
    val quantityGrams: String = "100",
    val unit: String = "gram",
    val expiryDateMillis: Long? = null, // Epoch millis cho DatePicker
    val source: String = "camera",
    val barcode: String = "",
    val isSaving: Boolean = false,
    val saveSuccess: Boolean = false,
    val errorMessage: String? = null,
    val isInitialized: Boolean = false
)

class FoodResultViewModel : ViewModel() {

    private val repository = PantryRepository()
    private val gson = Gson()

    private val _uiState = MutableStateFlow(FoodResultUiState())
    val uiState: StateFlow<FoodResultUiState> = _uiState.asStateFlow()

    /**
     * Khởi tạo từ kết quả AI/barcode (JSON string) hoặc empty cho manual.
     */
    fun initFromResult(
        resultJson: String?,
        source: String?,
        barcode: String?
    )

```



```

    ) {
        if (_uiState.value.isInitialized) return // Tránh init lại khi recompose

        val defaultExpiry = LocalDate.now().plusDays(7)
            .atStartOfDay(ZoneOffset.UTC).toInstant().toEpochMilli()

        if (resultJson != null) {
            try {
                val result = gson.fromJson(resultJson, FoodRecognitionResult::class.java)
                _uiState.update {
                    it.copy(
                        name = result.name,
                        caloriesPer100g = result.calories.toString(),
                        proteinPer100g = result.protein.toString(),
                        source = source ?: "camera",
                        barcode = barcode ?: "",
                        expiryDateMillis = defaultExpiry,
                        isInitialized = true
                    )
                }
            } catch (e: Exception) {
                // JSON parse failed → empty form
                _uiState.update {
                    it.copy(
                        source = source ?: "manual",
                        barcode = barcode ?: "",
                        expiryDateMillis = defaultExpiry,
                        isInitialized = true
                    )
                }
            }
        } else {
            // Manual entry
            _uiState.update {
                it.copy(
                    source = source ?: "manual",
                    barcode = barcode ?: "",
                    expiryDateMillis = defaultExpiry,
                    isInitialized = true
                )
            }
        }
    }

    // == FIELD UPDATES ==

    fun updateName(value: String) {
        _uiState.update { it.copy(name = value, errorMessage = null) }
    }

    fun updateCalories(value: String) {
        _uiState.update { it.copy(caloriesPer100g = value, errorMessage = null) }
    }

    fun updateProtein(value: String) {
        _uiState.update { it.copy(proteinPer100g = value, errorMessage = null) }
    }

    fun updateQuantity(value: String) {
        _uiState.update { it.copy(quantityGrams = value, errorMessage = null) }
    }

```

```

fun updateUnit(value: String) {
    _uiState.update { it.copy(unit = value, errorMessage = null) }
}

fun setExpiryDate(millis: Long) {
    _uiState.update { it.copy(expiryDateMillis = millis, errorMessage = null) }
}

/**
 * Kiểm tra form hợp lệ.
 */
fun isValid(): Boolean {
    val state = _uiState.value
    return state.name.isNotBlank()
        && (state.caloriesPer100g.toIntOrNull() ?: 0) >= 0
        && (state.proteinPer100g.toIntOrNull() ?: 0) >= 0
        && (state.quantityGrams.toIntOrNull() ?: 0) > 0
        && state.expiryDateMillis != null
}

/**
 * Lưu thực phẩm vào Firestore pantry collection.
 */
fun saveToKho() {
    val uid = repository.currentUid
    if (uid == null) {
        _uiState.update { it.copy(errorMessage = "Chưa đăng nhập") }
        return
    }

    val state = _uiState.value

    // Validate
    if (state.name.isBlank()) {
        _uiState.update { it.copy(errorMessage = "Vui lòng nhập tên thực phẩm") }
        return
    }
    val calories = state.caloriesPer100g.toIntOrNull()
    if (calories == null || calories < 0) {
        _uiState.update { it.copy(errorMessage = "Calo không hợp lệ") }
        return
    }
    val protein = state.proteinPer100g.toIntOrNull()
    if (protein == null || protein < 0) {
        _uiState.update { it.copy(errorMessage = "Protein không hợp lệ") }
        return
    }
    val quantity = state.quantityGrams.toIntOrNull()
    if (quantity == null || quantity <= 0) {
        _uiState.update { it.copy(errorMessage = "Số lượng phải lớn hơn 0") }
        return
    }
    if (state.expiryDateMillis == null) {
        _uiState.update { it.copy(errorMessage = "Vui lòng chọn ngày hết hạn") }
        return
    }

    // Tính expiry Timestamp
    val expiryTimestamp = Timestamp(
        java.util.Date(state.expiryDateMillis)
    )

```

```
// Tính status từ expiry date
val status = calculateExpiryStatus(expiryTimestamp)

val item = PantryItem(
    name = state.name.trim(),
    caloriesPer100g = calories,
    proteinPer100g = protein,
    quantityGrams = quantity,
    unit = state.unit,
    source = state.source,
    expiryDate = expiryTimestamp,
    status = status.firestoreValue,
    barcode = state.barcode
)

viewModelScope.launch {
    _uiState.update { it.copy(isSaving = true, errorMessage = null) }
    try {
        repository.addItem(uid, item)
        _uiState.update { it.copy(isSaving = false, saveSuccess = true) }
    } catch (e: Exception) {
        _uiState.update {
            it.copy(
                isSaving = false,
                errorMessage = "Lưu thất bại: ${e.message}"
            )
        }
    }
}

/** Clear error */
fun clearError() {
    _uiState.update { it.copy(errorMessage = null) }
}
}
```

File: pantry/viewmodel/PantryListViewModel.kt

```

package com.team.smartnutrition.pantry.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.team.smartnutrition.pantry.data.PantryRepository
import com.team.smartnutrition.pantry.model.PantryItem
import com.team.smartnutrition.pantry.util.ExpiryStatus
import com.team.smartnutrition.pantry.util.calculateExpiryStatus
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.catch
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * PANTRY LIST VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Load realtime danh sách thực phẩm từ Firestore
 * - Lọc theo trạng thái hạn sử dụng (Tất cả / Sắp hết hạn / Đã hết hạn)
 * - Tìm kiếm theo tên
 * - Xóa thực phẩm
 * - Tự động cập nhật status khi hạn sử dụng thay đổi
 */

data class PantryListUiState(
    val items: List<PantryItem> = emptyList(),
    val filteredItems: List<PantryItem> = emptyList(),
    val selectedFilter: ExpiryFilter = ExpiryFilter.ALL,
    val searchQuery: String = "",
    val isLoading: Boolean = true,
    val errorMessage: String? = null,
    val showDeleteDialog: Boolean = false,
    val itemToDelete: PantryItem? = null,
    val showAddSheet: Boolean = false
)

enum class ExpiryFilter(val label: String) {
    ALL("Tất cả"),
    EXPIRING("Sắp hết hạn"),
    EXPIRED("Đã hết hạn")
}

class PantryListViewModel : ViewModel() {

    private val repository = PantryRepository()

    private val _uiState = MutableStateFlow(PantryListUiState())
    val uiState: StateFlow<PantryListUiState> = _uiState.asStateFlow()

    init {
        loadItems()
    }

    /**
     * Load realtime danh sách từ Firestore.
     * Tự cập nhật status nếu cần.
     */
    private fun loadItems() {

```

```

        val uid = repository.currentUid ?: return

        viewModelScope.launch {
            repository.getItems(uid)
                .catch { e ->
                    _uiState.update {
                        it.copy(isLoading = false, errorMessage = "Lỗi tải kho: ${e.message}")
                    }
                }
            .collect { items ->
                // Tự cập nhật status cho items có hạn sử dụng thay đổi
                val updatedItems = refreshStatuses(uid, items)

                _uiState.update { state ->
                    state.copy(
                        items = updatedItems,
                        filteredItems = applyFilters(
                            updatedItems, state.selectedFilter, state.searchQuery
                        ),
                        isLoading = false,
                        errorMessage = null
                    )
                }
            }
        }

    /**
     * Đặt bộ lọc trạng thái hạn sử dụng.
     */
    fun setFilter(filter: ExpiryFilter) {
        _uiState.update { state ->
            state.copy(
                selectedFilter = filter,
                filteredItems = applyFilters(state.items, filter, state.searchQuery)
            )
        }
    }

    /**
     * Cập nhật query tìm kiếm.
     */
    fun setSearchQuery(query: String) {
        _uiState.update { state ->
            state.copy(
                searchQuery = query,
                filteredItems = applyFilters(state.items, state.selectedFilter, query)
            )
        }
    }

    /**
     * Hiện confirm dialog xóa.
     */
    fun showDeleteConfirm(item: PantryItem) {
        _uiState.update { it.copy(showDeleteDialog = true, itemToDelete = item) }
    }

    /**
     * Ẩn confirm dialog xóa.
     */
    fun dismissDeleteConfirm() {

```

```

        _uiState.update { it.copy(showDeleteDialog = false, itemToDelete = null) }
    }

    /**
     * Xóa thực phẩm khỏi kho.
     */
    fun deleteItem(item: PantryItem) {
        val uid = repository.currentUid ?: return

        viewModelScope.launch {
            try {
                repository.deleteItem(uid, item.id)
                _uiState.update { it.copy(showDeleteDialog = false, itemToDelete = null) }
            } catch (e: Exception) {
                _uiState.update {
                    it.copy(
                        errorMessage = "Không thể xóa: ${e.message}",
                        showDeleteDialog = false,
                        itemToDelete = null
                    )
                }
            }
        }
    }

    /** Toggle hiển thị BottomSheet thêm mới */
    fun toggleAddSheet(show: Boolean) {
        _uiState.update { it.copy(showAddSheet = show) }
    }

    /** Clear error message */
    fun clearError() {
        _uiState.update { it.copy(errorMessage = null) }
    }

    // == PRIVATE HELPERS ==

    /**
     * Áp dụng bộ lọc + tìm kiếm lên danh sách items.
     */
    private fun applyFilters(
        items: List<PantryItem>,
        filter: ExpiryFilter,
        searchQuery: String
    ): List<PantryItem> {
        var result = items

        // Lọc theo trạng thái
        result = when (filter) {
            ExpiryFilter.ALL -> result
            ExpiryFilter.EXPIRING -> result.filter {
                calculateExpiryStatus(it.expiryDate) == ExpiryStatus.EXPIRING
            }
            ExpiryFilter.EXPIRED -> result.filter {
                calculateExpiryStatus(it.expiryDate) == ExpiryStatus.EXPIRED
            }
        }

        // Lọc theo tìm kiếm (case-insensitive)
        if (searchQuery.isNotBlank()) {
            result = result.filter {
                it.name.contains(searchQuery, ignoreCase = true)
            }
        }
    }

```

```
    }  
  }  
  
  return result  
}  
  
/**  
 * Tự cập nhật status nếu hạn sử dụng thay đổi so với Firestore.  
 * VD: item ngày hôm qua còn "fresh", hôm nay thành "expiring".  
 */  
private fun refreshStatuses(uid: String, items: List<PantryItem>): List<PantryItem> {  
    return items.map { item ->  
        val currentStatus = calculateExpiryStatus(item.expiryDate)  
        if (currentStatus.firestoreValue != item.status) {  
            // Update Firestore async (fire-and-forget)  
            viewModelScope.launch {  
                try {  
                    repository.updateItem(  
                        uid, item.id,  
                        mapOf("status" to currentStatus.firestoreValue)  
                    )  
                } catch (_: Exception) {  
                    // Silently ignore – sẽ retry lần load tiếp  
                }  
            }  
            item.copy(status = currentStatus.firestoreValue)  
        } else {  
            item  
        }  
    }  
}
```